ANDROID_CLIENT_DESIGN

Helix OTA — Android 15 Client Architecture Document

Project: Helix OTA System
Version: 1.0.0-MVP
Target: Android 15 on Orange Pi 5 Max (Rockchip RK3588)
Status: Design Document

Table of Contents

---

1. Android 15 OTA Overview

1.1 How Android OTA Works

Android Over-The-Air (OTA) updates are the mechanism by which the operating system, firmware, and vendor partitions are updated without requiring a physical connection. In Android 15, the OTA pipeline is built on the **Virtual A/B** (also called "Seamless Updates v2") architecture, which combines the safety of A/B partitions with the storage efficiency of snapshot-based merging.

At a high level, an OTA update follows this lifecycle:

1. **Update Discovery** — The device checks an update server (or receives a push notification) indicating a new build is available.

2. **Payload Download** — The OTA payload (a ZIP archive containing `payload.bin`) is downloaded to the device's `data` partition.
3. **Payload Verification** — The payload's SHA-256 hash and RSA signature are verified against trusted keys built into the system image.
4. **Payload Application** — The `update_engine` daemon applies the payload to the inactive slot's partitions. In Virtual A/B, this involves writing to COW (Copy-on-Write) snapshots via `dm-snapshot` and `dm-user` device-mapper targets.
5. **Post-Install Verification** — The newly written partitions are mounted and verified (e.g., dm-verity checks, boot image integrity).
6. **Slot Switch & Reboot** — The boot control HAL marks the inactive slot as active, and the device reboots into the new slot.
7. **Merge Phase** — After a successful boot, the COW snapshots are merged into the underlying partitions. If the device fails to boot, it automatically falls back to the old slot.
8. **Commit or Rollback** — After merge completion, the update is committed. If the merge fails or the device reboots too many times, the boot control HAL triggers a rollback.

1.2 Virtual A/B Architecture

Virtual A/B was introduced in Android 11 and is mandatory for all new devices launching with Android 13+. The RK3588 platform targeting Android 15 must implement Virtual A/B. Key characteristics:

- **Storage Efficiency:** Unlike classic A/B where both slots maintain complete copies of every partition, Virtual A/B keeps only one complete copy. The inactive slot's partitions are overlaid with COW snapshots during update application. This reduces the storage overhead from ~2x to ~1x plus a temporary COW file.
- **Snapshot-Based Safety:** Updates are written to COW files stored in `data`. The original partition data is preserved until the merge phase completes successfully. If anything goes wrong, the device simply discards the COW file and boots from the original data.
- **`dm-user` Daemon:** Android 15 uses the `dm-user` device-mapper target (replacing the older `dm-snapshot` + `dm-snapshot-merge` approach). The `snapuserd` daemon manages the COW file I/O, presenting a block device to `update_engine` for writing while simultaneously serving reads from either the base partition or the COW file.
- **Super Partition:** All A/B-enabled dynamic partitions (system, vendor, product, odm, etc.) reside within a single `super` partition. The `super` partition contains two logical partition groups: `super_a` and `super_b`, managed by `liblp` (logical partition library).

The Virtual A/B flow for a single partition looks like:

```
[super partition]
  ├── super_a (logical group, active)
  │    ├── system_a  (base + COW overlay during merge)
  │    ├── vendor_a
  │    └── ...
  └── super_b (logical group, inactive)
       ├── system_b  (COW target during update)
       ├── vendor_b
```

```
    └── ...

/data/ota/
  └── system_b.cow  (COW file written by update_engine via
snapuserd)
```

1.3 update_engine Daemon

`update_engine` is the system daemon responsible for applying OTA payloads. It runs as a privileged process (`com.android.otatest`) and exposes a Binder IPC interface for clients to control the update process.

Key responsibilities of `update_engine`:

- **Payload Parsing:** Reads the payload header and protobuf manifest from `payload.bin` to determine which partitions to update and the operations (REPLACE, REPLACE_BZ, ZERO, SOURCE_COPY, SOURCE_BROTLI) to perform.
- **Partition Writing:** For each partition operation, writes data to the target partition (or COW file in Virtual A/B mode) via `dm-user`.
- **Progress Reporting:** Emits progress updates (0–100%) and status changes via the `IUpdateEngineCallback` Binder interface.
- **Post-Install Execution:** After all partition operations complete, runs post-install steps (e.g., `postinst` scripts, dm-verity setup for the new slot).
- **Error Handling:** If any operation fails, `update_engine` reports the error code and does not switch slots. The client must handle retry or rollback.

`update_engine` reads its input from: 1. The **payload file** (`payload.bin`) — the actual binary data containing partition images. 2. The **payload properties file** (`payload_properties.txt`) — key-value pairs that tell `update_engine` how to parse the payload (offset, size, metadata size, etc.).

The client is responsible for downloading the OTA zip, extracting these files, and passing their paths (along with offset/size metadata) to `update_engine` via the `applyPayload()` Binder call.

---

2. Client Architecture

2.1 HelixOtaClient Android App

The `HelixOtaClient` is a **system-privileged application** pre-installed in `/system/priv-app/HelixOtaClient/`. As a privileged system app, it has:

- Signature-level permissions to interact with `update_engine` via Binder IPC.
- Access to protected storage paths (`/data/ota/`, `/cache/`).
- The ability to run foreground services without user-visible restrictions.
- Access to `REBOOT` permission for scheduling reboots.

The app is composed of the following major components:

```
com.helix.ota.client/
├── HelixOtaApplication.kt          # Application subclass, DI
setup
├── di/
│   ├── AppModule.kt                # Singleton bindings
│   ├── NetworkModule.kt            # OkHttp/Retrofit instances
│   └── DatabaseModule.kt           # Room database
├── api/
│   ├── HelixOtaApi.kt              # Retrofit interface to Helix
server
│   └── models/
│       ├── UpdateInfo.kt           # Server response models
│       ├── DownloadInfo.kt
│       └── ReportRequest.kt
├── engine/
│   ├── UpdateEngineProxy.kt        # Binder IPC wrapper for
update_engine
│   ├── UpdateEngineCallback.kt     # Callback implementation
│   └── BootControlHelper.kt        # Boot control HAL interaction
├── service/
│   ├── UpdateCheckService.kt       # WorkManager periodic job
│   ├── DownloadService.kt          # Foreground download service
│   ├── VerificationService.kt      # Integrity verification
│   ├── InstallService.kt           # update_engine orchestration
│   └── ReportingService.kt         # Status reporting to server
├── download/
│   ├── ChunkedDownloader.kt        # HTTP Range-based downloader
│   ├── DownloadStorageManager.kt   # Disk space & file management
│   └── NetworkPolicyManager.kt     # WiFi/cellular policy
├── verify/
│   ├── ZipVerifier.kt              # ZIP structural integrity
│   ├── HashVerifier.kt             # SHA-256 hash verification
│   ├── SignatureVerifier.kt        # RSA signature verification
│   └── DeviceCompatibilityCheck.kt # Device/version validation
├── notification/
│   └── NotificationHelper.kt       # User-facing notifications
├── state/
│   ├── OtaStateMachine.kt          # Central state machine
│   └── OtaState.kt                 # State & event definitions
├── database/
│   ├── OtaDatabase.kt              # Room database
│   ├── UpdateDao.kt                # DAO for update records
│   └── entities/
│       └── UpdateRecord.kt         # Persisted update record
├── receiver/
│   ├── BootCompletedReceiver.kt    # Schedule checks after reboot
│   └── NetworkChangeReceiver.kt    # Resume downloads on
connectivity
└── ui/
    ├── MainActivity.kt             # Settings & manual check UI
    ├── UpdateAvailableActivity.kt  # Update confirmation dialog
    └── InstallProgressActivity.kt  # Installation progress
```

2.2 Component Details

2.2.1 UpdateCheckService

A **WorkManager** periodic worker that runs on a configurable interval (default: every 4 hours). It queries the Helix OTA server for available updates.

```
class UpdateCheckService(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    @Inject lateinit var helixApi: HelixOtaApi
    @Inject lateinit var stateMachine: OtaStateMachine
    @Inject lateinit var prefs: OtaPreferences

    override suspend fun doWork(): Result {
        if (stateMachine.currentState != OtaState.IDLE) {
            return Result.success() // Already processing an
        update
        }

        stateMachine.transition(OtaEvent.CHECK_START)

        return try {
            val request = CheckUpdateRequest(
                deviceId = prefs.deviceId,
                hardwareModel = Build.HARDWARE,         // "rk3588"
                currentBuild = Build.DISPLAY,           // e.g.,
        "TP1A.240305.001"
                currentVersion = prefs.installedVersion, // e.g.,
        "1.0.0-mvp"
                buildFingerprint = Build.FINGERPRINT,
                securityPatchLevel = Build.VERSION.SECURITY_PATCH,
                batteryLevel = getBatteryLevel(),
                availableStorage = getAvailableStorage()
            )

            val response = helixApi.checkForUpdate(request)

            when {
                response.updateAvailable -> {
                    prefs.cachedUpdateInfo = response.updateInfo
                    stateMachine.transition(OtaEvent.UPDATE_FOUND)
                    NotificationHelper.showUpdateAvailable(
                        applicationContext,
                        response.updateInfo.version,
                        response.updateInfo.sizeBytes,
                        response.updateInfo.changelog
                    )
                }
                else -> {
                    stateMachine.transition(OtaEvent.NO_UPDATE)
```

```
                }
            }
            Result.success()
        } catch (e: Exception) {
            stateMachine.transition(OtaEvent.CHECK_FAILED)
            Result.retry()
        }
    }

    companion object {
        private const val UNIQUE_WORK_NAME =
        "helix_ota_update_check"

        fun schedule(context: Context, intervalHours: Long = 4) {
            val request =
        PeriodicWorkRequestBuilder<UpdateCheckService>(
                intervalHours, TimeUnit.HOURS
            )
                .setConstraints(
                    Constraints.Builder()
                        .setRequiredNetworkType(NetworkType.CONNECTED)
                        .setRequiresBatteryNotLow(true)
                        .build()
                )
                .setBackoffCriteria(
                    BackoffPolicy.EXPONENTIAL,
                    WorkRequest.MIN_BACKOFF_MILLIS,
                    TimeUnit.MILLISECONDS
                )
                .build()

            WorkManager.getInstance(context)
                .enqueueUniquePeriodicWork(
                    UNIQUE_WORK_NAME,
                    ExistingPeriodicWorkPolicy.KEEP,
                    request
                )
        }
    }
}
```

2.2.2 DownloadService

A **foreground service** that manages the resumable, chunked download of the OTA zip. It posts a persistent notification showing download progress.

```
class DownloadService : Service() {

    @Inject lateinit var downloader: ChunkedDownloader
    @Inject lateinit var storageManager: DownloadStorageManager
    @Inject lateinit var stateMachine: OtaStateMachine
    @Inject lateinit var prefs: OtaPreferences
```

```
private val binder = LocalBinder()
private var downloadJob: Job? = null

inner class LocalBinder : Binder() {
    fun getService(): DownloadService = this@DownloadService
}

override fun onBind(intent: Intent?) = binder

override fun onStartCommand(intent: Intent?, flags: Int,
    startId: Int): Int {
    when (intent?.action) {
        ACTION_START -> startDownload(intent)
        ACTION_PAUSE -> pauseDownload()
        ACTION_RESUME -> resumeDownload()
        ACTION_CANCEL -> cancelDownload()
    }
    return START_STICKY
}

private fun startDownload(intent: Intent) {
    val updateInfo = prefs.cachedUpdateInfo ?: return
    val downloadUrl = updateInfo.downloadUrl
    val targetFile = storageManager.getOtaZipFile()

    startForeground(NOTIFICATION_ID,

    NotificationHelper.buildDownloadProgressNotification(this,
    0, 0L, 0L))

    stateMachine.transition(OtaEvent.DOWNLOAD_START)

    downloadJob = CoroutineScope(Dispatchers.IO).launch {
        downloader.download(
            url = downloadUrl,
            targetFile = targetFile,
            headers = mapOf(
                "X-Device-ID" to prefs.deviceId,
                "X-OTA-Version" to updateInfo.version
            ),
            listener = object : DownloadProgressListener {
                override fun onProgress(downloaded: Long,
    total: Long, speedBps: Long) {
                    val percent = ((downloaded.toDouble() /
    total) * 100).toInt()
                    updateNotification(percent, downloaded,
    total)
                    stateMachine.transition(
                        OtaEvent.DOWNLOAD_PROGRESS(percent,
    downloaded, total)
                    )
                }
                override fun onComplete(file: File) {
```

```
            stateMachine.transition(OtaEvent.DOWNLOAD_COMPLETE)
                        stopForeground(STOP_FOREGROUND_REMOVE)
                        // Automatically proceed to verification
                        startVerificationService()
                    }
                    override fun onError(error: DownloadError) {

            stateMachine.transition(OtaEvent.DOWNLOAD_FAILED(error))
                        stopForeground(STOP_FOREGROUND_REMOVE)

            NotificationHelper.showDownloadError(this@DownloadService,
            error)
                    }
                }
            )
        }
    }

    private fun pauseDownload() {
        downloadJob?.cancel()
        downloader.pause()
        stateMachine.transition(OtaEvent.DOWNLOAD_PAUSED)
    }

    private fun resumeDownload() {
        // Reuse startDownload logic; ChunkedDownloader detects
        partial file
        startDownload(Intent(this,
        DownloadService::class.java).apply {
            action = ACTION_START
        })
    }

    private fun cancelDownload() {
        downloadJob?.cancel()
        downloader.cancel()
        storageManager.deletePartialFiles()
        stateMachine.transition(OtaEvent.DOWNLOAD_CANCELLED)
        stopForeground(STOP_FOREGROUND_REMOVE)
        stopSelf()
    }

    companion object {
        const val ACTION_START =
        "com.helix.ota.action.DOWNLOAD_START"
        const val ACTION_PAUSE =
        "com.helix.ota.action.DOWNLOAD_PAUSE"
        const val ACTION_RESUME =
        "com.helix.ota.action.DOWNLOAD_RESUME"
        const val ACTION_CANCEL =
        "com.helix.ota.action.DOWNLOAD_CANCEL"
        const val NOTIFICATION_ID = 1001
```

```
    }
}
```

2.2.3 VerificationService

An **IntentService** that performs multi-step verification of the downloaded OTA zip. It runs on a background thread and reports results via the state machine.

```
class VerificationService : IntentService("HelixOtaVerification")
        {

    @Inject lateinit var zipVerifier: ZipVerifier
    @Inject lateinit var hashVerifier: HashVerifier
    @Inject lateinit var signatureVerifier: SignatureVerifier
    @Inject lateinit var compatibilityCheck:
        DeviceCompatibilityCheck
    @Inject lateinit var stateMachine: OtaStateMachine
    @Inject lateinit var prefs: OtaPreferences

    override fun onHandleIntent(intent: Intent?) {
        stateMachine.transition(OtaEvent.VERIFY_START)

        val otaZip = File(prefs.otaZipPath)
        val updateInfo = prefs.cachedUpdateInfo ?: run {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.MISSING_UPDATE_
            return
        }

        try {
            // Step 1: ZIP structural integrity
            val zipResult = zipVerifier.verify(otaZip)
            if (!zipResult.isValid) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.ZIP_CORRUPT))
                return
            }

            // Step 2: SHA-256 hash verification
            val hashResult = hashVerifier.verify(
                file = otaZip,
                expectedHash = updateInfo.sha256Hash,
                hashFile = File(otaZip.parent, "$
        {otaZip.name}.sha256")
            )
            if (!hashResult) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.HASH_MISMATCH))
                return
            }

            // Step 3: RSA signature verification on payload
        manifest
```

```
            val sigResult =
        signatureVerifier.verifyPayloadManifest(
                otaZip = otaZip,
                publicKey = loadOtaPublicKey()
            )
            if (!sigResult) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.SIGNATURE_INVAL
                return
            }

            // Step 4: payload_properties.txt format validation
            val propsResult = verifyPayloadProperties(otaZip)
            if (!propsResult) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.PROPERTIES_INVA
                return
            }

            // Step 5: Device compatibility
            val compatResult = compatibilityCheck.verify(
                targetHardware = updateInfo.targetHardware,
                minVersion = updateInfo.minCurrentVersion,
                targetVersion = updateInfo.version
            )
            if (!compatResult) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.DEVICE_INCOMPAT
                return
            }

            stateMachine.transition(OtaEvent.VERIFY_COMPLETE)
        } catch (e: Exception) {

        stateMachine.transition(OtaEvent.VERIFY_FAILED(VerifyError.VERIFICATION_EX
        }
    }
}
```

2.2.4 InstallService

A **foreground service** that orchestrates the `update_engine` payload application. This is the most critical component as it interfaces directly with the system's OTA engine.

```
class InstallService : Service() {

    @Inject lateinit var updateEngine: UpdateEngineProxy
    @Inject lateinit var stateMachine: OtaStateMachine
    @Inject lateinit var prefs: OtaPreferences
    @Inject lateinit var reportingService: ReportingService
```

```
private val engineCallback = object :
    IUpdateEngineCallback.Stub() {
    override fun onStatusUpdate(status: Int, percentage:
    Float) {
        val state = UpdateEngineStatus.fromCode(status)
        when (state) {
            UpdateEngineStatus.UPDATE_STATUS_IDLE -> { /* no-
    op */ }

    UpdateEngineStatus.UPDATE_STATUS_CHECKING_FOR_UPDATE ->
    { /* no-op */ }
            UpdateEngineStatus.UPDATE_STATUS_UPDATE_AVAILABLE
    -> { /* no-op */ }
            UpdateEngineStatus.UPDATE_STATUS_DOWNLOADING -> {
                val percent = (percentage * 100).toInt()

    stateMachine.transition(OtaEvent.INSTALL_PROGRESS(percent))
                updateInstallNotification(percent)
                reportingService.reportProgress(percent)
            }
            UpdateEngineStatus.UPDATE_STATUS_VERIFYING -> {

    stateMachine.transition(OtaEvent.INSTALL_VERIFYING)
            }
            UpdateEngineStatus.UPDATE_STATUS_FINALIZING -> {

    stateMachine.transition(OtaEvent.INSTALL_FINALIZING)
            }

    UpdateEngineStatus.UPDATE_STATUS_UPDATED_NEED_REBOOT -> {

    stateMachine.transition(OtaEvent.INSTALL_NEEDS_REBOOT)
                reportingService.reportStatus("needs_reboot")
                showRebootNotification()
            }
            UpdateEngineStatus.UPDATE_STATUS_REPORTING_ERROR
    -> {

    stateMachine.transition(OtaEvent.INSTALL_ENGINE_ERROR(
                    EngineError.REPORTING_ERROR))
            }
            else -> { /* unknown status */ }
        }
    }

    override fun onPayloadApplicationComplete(errorCode: Int)
    {
        if (errorCode == UpdateEngineError.SUCCESS.code) {
            stateMachine.transition(OtaEvent.INSTALL_COMPLETE)
            reportingService.reportStatus("install_complete")
        } else {
            val error = UpdateEngineError.fromCode(errorCode)

    stateMachine.transition(OtaEvent.INSTALL_ENGINE_ERROR(error))
```

```
                reportingService.reportStatus("install_failed",
        error.description)

        NotificationHelper.showInstallError(this@InstallService,
        error)
            }
        }
    }

    override fun onStartCommand(intent: Intent?, flags: Int,
        startId: Int): Int {
        when (intent?.action) {
            ACTION_INSTALL -> startInstallation()
            ACTION_SUSPEND -> updateEngine.suspend()
            ACTION_RESUME -> updateEngine.resume()
            ACTION_CANCEL -> {
                updateEngine.cancel()

        stateMachine.transition(OtaEvent.INSTALL_CANCELLED)
            }
        }
        return START_STICKY
    }

    private fun startInstallation() {
        startForeground(NOTIFICATION_ID,

        NotificationHelper.buildInstallProgressNotification(this,
        0))

        stateMachine.transition(OtaEvent.INSTALL_START)

        val otaZip = File(prefs.otaZipPath)
        val extractDir = File(otaZip.parent, "extracted").also {
        it.mkdirs() }

        // Extract payload.bin and payload_properties.txt from the
        OTA zip
        extractPayloadFiles(otaZip, extractDir)

        val payloadBin = File(extractDir, "payload.bin")
        val payloadProps = File(extractDir,
        "payload_properties.txt")

        // Parse payload_properties.txt for offset and size
        val props = parsePayloadProperties(payloadProps)

        // Bind to update_engine and apply the payload
        updateEngine.bind(engineCallback)
        updateEngine.applyPayload(
            url = "file://${payloadBin.absolutePath}",
            offset = props.offset,
            size = props.size,
            headers = props.headers
```

```
            )
        }

        private fun parsePayloadProperties(file: File):
            PayloadProperties {
            val lines = file.readLines().associate {
                val (key, value) = it.split("=", limit = 2)
                key.trim() to value.trim()
            }
            return PayloadProperties(
                offset = lines["FILE_HASH"]?.let { 0L } ?: 0L,
                size = lines["FILE_SIZE"]?.toLongOrNull() ?: 0L,
                metadataSize =
            lines["METADATA_SIZE"]?.toLongOrNull() ?: 0L,
                headers = arrayOf(
                    "FILE_HASH=${lines["FILE_HASH"]}",
                    "FILE_SIZE=${lines["FILE_SIZE"]}",
                    "METADATA_HASH=${lines["METADATA_HASH"]}",
                    "METADATA_SIZE=${lines["METADATA_SIZE"]}"
                )
            )
        }

        companion object {
            const val ACTION_INSTALL = "com.helix.ota.action.INSTALL"
            const val ACTION_SUSPEND =
            "com.helix.ota.action.INSTALL_SUSPEND"
            const val ACTION_RESUME =
            "com.helix.ota.action.INSTALL_RESUME"
            const val ACTION_CANCEL =
            "com.helix.ota.action.INSTALL_CANCEL"
            const val NOTIFICATION_ID = 1002
        }
    }

    data class PayloadProperties(
        val offset: Long,
        val size: Long,
        val metadataSize: Long,
        val headers: Array<String>
    )
```

2.2.5 ReportingService

Reports update status back to the Helix OTA server for fleet management and analytics.

```
    class ReportingService @Inject constructor(
        private val helixApi: HelixOtaApi,
        private val prefs: OtaPreferences
    ) {
        suspend fun reportStatus(status: String, errorDetail: String?
            = null) {
```

```
            try {
                helixApi.reportUpdateStatus(ReportRequest(
                    deviceId = prefs.deviceId,
                    updateId = prefs.cachedUpdateInfo?.updateId ?: "",
                    status = status,
                    errorDetail = errorDetail,
                    timestamp = System.currentTimeMillis(),
                    currentSlot = BootControlHelper.getCurrentSlot(),
                    bootCount = prefs.bootCountSinceUpdate
                ))
            } catch (e: Exception) {
                // Report failures are non-critical; log and continue
                Log.w("ReportingService", "Failed to report status",
            e)
            }
        }

        suspend fun reportProgress(percent: Int) {
            // Throttle progress reports to at most once per 5%
            if (percent % 5 != 0) return
            reportStatus("installing_$percent%")
        }

        suspend fun reportCheckResult(updateAvailable: Boolean,
            version: String?) {
            helixApi.reportCheckResult(CheckReportRequest(
                deviceId = prefs.deviceId,
                updateAvailable = updateAvailable,
                serverVersion = version,
                timestamp = System.currentTimeMillis()
            ))
        }
    }
```

2.2.6 NotificationHelper

Manages all user-facing notifications with appropriate channels and styling.

```
    object NotificationHelper {

        private const val CHANNEL_UPDATES = "helix_ota_updates"
        private const val CHANNEL_DOWNLOAD = "helix_ota_download"
        private const val CHANNEL_INSTALL = "helix_ota_install"

        fun createChannels(context: Context) {
            val manager = NotificationManagerCompat.from(context)
            manager.createNotificationChannels(listOf(
                NotificationChannel(
                    CHANNEL_UPDATES,
                    "Update Notifications",
                    NotificationManager.IMPORTANCE_HIGH
                ).apply {
```

```
            description = "Notifications for available system
        updates"
            enableVibration(true)
        },
        NotificationChannel(
            CHANNEL_DOWNLOAD,
            "Download Progress",
            NotificationManager.IMPORTANCE_LOW
        ).apply {
            description = "OTA download progress indicator"
            setShowBadge(false)
        },
        NotificationChannel(
            CHANNEL_INSTALL,
            "Installation Progress",
            NotificationManager.IMPORTANCE_LOW
        ).apply {
            description = "System update installation
        progress"
            setShowBadge(false)
        }
    ))
}

fun showUpdateAvailable(
    context: Context,
    version: String,
    sizeBytes: Long,
    changelog: String
) {
    val sizeStr = Formatter.formatFileSize(context, sizeBytes)
    val intent = Intent(context,
    UpdateAvailableActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_NEW_TASK
    }
    val pendingIntent = PendingIntent.getActivity(
        context, 0, intent,
        PendingIntent.FLAG_UPDATE_CURRENT or
    PendingIntent.FLAG_IMMUTABLE
    )

    val notification = NotificationCompat.Builder(context,
    CHANNEL_UPDATES)
        .setSmallIcon(R.drawable.ic_system_update)
        .setContentTitle("System update available")
        .setContentText("Android $version ($sizeStr)")
        .setStyle(
            NotificationCompat.BigTextStyle()
                .bigText("Version $version ($sizeStr)
    \n\n$changelog")
        )
        .setContentIntent(pendingIntent)
        .setAutoCancel(true)
        .addAction(
```

```
                R.drawable.ic_download,
                "Download",
                createDownloadAction(context)
            )
            .addAction(
                R.drawable.ic_later,
                "Later",
                createDismissAction(context)
            )
            .build()

        NotificationManagerCompat.from(context)
            .notify(NOTIFICATION_UPDATE_AVAILABLE, notification)
    }

    fun buildDownloadProgressNotification(
        context: Context,
        percent: Int,
        downloaded: Long,
        total: Long
    ): Notification {
        return NotificationCompat.Builder(context,
        CHANNEL_DOWNLOAD)
            .setSmallIcon(R.drawable.ic_download)
            .setContentTitle("Downloading system update")
            .setContentText("${percent}% — $
        {Formatter.formatFileSize(context, downloaded)} / $
        {Formatter.formatFileSize(context, total)}")
            .setProgress(100, percent, false)
            .setOngoing(true)
            .addAction(R.drawable.ic_pause, "Pause",
        createPauseAction(context))
            .addAction(R.drawable.ic_cancel, "Cancel",
        createCancelAction(context))
            .build()
    }

    fun showRebootNotification() { /* ... */ }
    fun showInstallError(context: Context, error:
        UpdateEngineError) { /* ... */ }
    fun showDownloadError(context: Context, error: DownloadError)
        { /* ... */ }
}
```

2.3 State Machine

The OTA process is modeled as a strict finite state machine to prevent invalid transitions and ensure consistency.

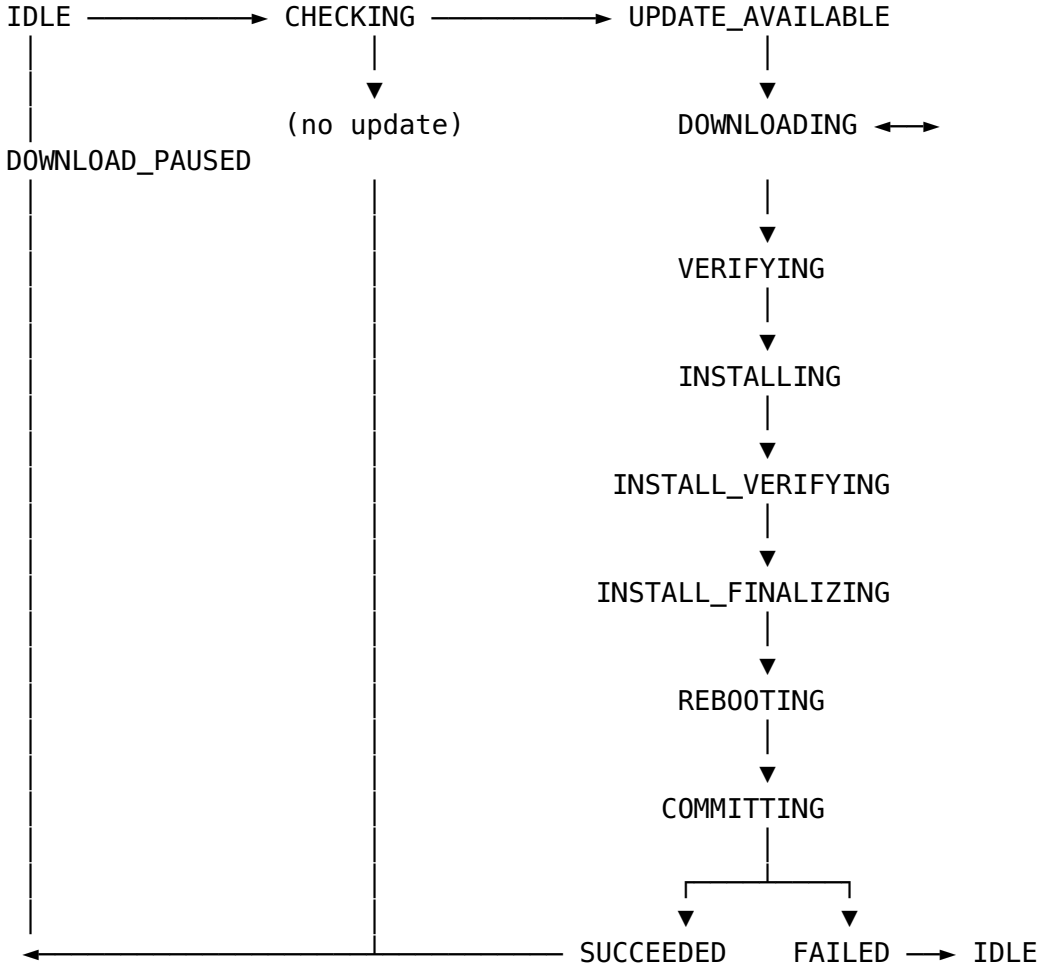
```
enum class OtaState {
    IDLE,
    CHECKING,
    UPDATE_AVAILABLE,
```

```
    DOWNLOADING,
    DOWNLOAD_PAUSED,
    VERIFYING,
    INSTALLING,
    INSTALL_VERIFYING,
    INSTALL_FINALIZING,
    REBOOTING,
    COMMITTING,
    SUCCEEDED,
    FAILED
}

sealed class OtaEvent {
    object CheckStart : OtaEvent()
    object UpdateFound : OtaEvent()
    object NoUpdate : OtaEvent()
    object CheckFailed : OtaEvent()
    object DownloadStart : OtaEvent()
    data class DownloadProgress(val percent: Int, val downloaded:
        Long, val total: Long) : OtaEvent()
    object DownloadComplete : OtaEvent()
    object DownloadPaused : OtaEvent()
    object DownloadCancelled : OtaEvent()
    data class DownloadFailed(val error: DownloadError) :
        OtaEvent()
    object VerifyStart : OtaEvent()
    object VerifyComplete : OtaEvent()
    data class VerifyFailed(val error: VerifyError) : OtaEvent()
    object InstallStart : OtaEvent()
    data class InstallProgress(val percent: Int) : OtaEvent()
    object InstallVerifying : OtaEvent()
    object InstallFinalizing : OtaEvent()
    object InstallNeedsReboot : OtaEvent()
    object InstallComplete : OtaEvent()
    data class InstallEngineError(val error: UpdateEngineError) :
        OtaEvent()
    object InstallCancelled : OtaEvent()
    object RebootStart : OtaEvent()
    object CommitStart : OtaEvent()
    object Succeeded : OtaEvent()
    data class Failed(val reason: String) : OtaEvent()
}

class OtaStateMachine @Inject constructor() {
    private val _state = MutableStateFlow(OtaState.IDLE)
    val state: StateFlow<OtaState> = _state.asStateFlow()

    private val validTransitions = mapOf<OtaState, Set<OtaState>>(
        OtaState.IDLE to setOf(OtaState.CHECKING,
        OtaState.FAILED),
        OtaState.CHECKING to setOf(OtaState.UPDATE_AVAILABLE,
        OtaState.IDLE, OtaState.FAILED),
        OtaState.UPDATE_AVAILABLE to setOf(OtaState.DOWNLOADING,
        OtaState.IDLE),
```

```
        OtaState.DOWNLOADING to setOf(OtaState.DOWNLOAD_PAUSED,
        OtaState.VERIFYING, OtaState.FAILED),
        OtaState.DOWNLOAD_PAUSED to setOf(OtaState.DOWNLOADING,
        OtaState.IDLE),
        OtaState.VERIFYING to setOf(OtaState.INSTALLING,
        OtaState.FAILED),
        OtaState.INSTALLING to setOf(OtaState.INSTALL_VERIFYING,
        OtaState.INSTALL_FINALIZING, OtaState.FAILED),
        OtaState.INSTALL_VERIFYING to
        setOf(OtaState.INSTALL_FINALIZING, OtaState.FAILED),
        OtaState.INSTALL_FINALIZING to setOf(OtaState.REBOOTING,
        OtaState.FAILED),
        OtaState.REBOOTING to setOf(OtaState.COMMITTING,
        OtaState.FAILED),
        OtaState.COMMITTING to setOf(OtaState.SUCCEEDED,
        OtaState.FAILED),
        OtaState.SUCCEEDED to setOf(OtaState.IDLE),
        OtaState.FAILED to setOf(OtaState.IDLE)
    )

    fun transition(event: OtaEvent) {
        val newState = mapEventToState(event)
        val current = _state.value
        if (validTransitions[current]?.contains(newState) ==
        true) {
            _state.value = newState
        } else {
            Log.e("OtaStateMachine",
                "Invalid transition: $current -> $newState
        (event: $event)")
        }
    }

    val currentState: OtaState get() = _state.value

    private fun mapEventToState(event: OtaEvent): OtaState = when
        (event) {
        is OtaEvent.CheckStart -> OtaState.CHECKING
        is OtaEvent.UpdateFound -> OtaState.UPDATE_AVAILABLE
        is OtaEvent.NoUpdate -> OtaState.IDLE
        is OtaEvent.CheckFailed -> OtaState.FAILED
        is OtaEvent.DownloadStart -> OtaState.DOWNLOADING
        is OtaEvent.DownloadProgress -> OtaState.DOWNLOADING
        is OtaEvent.DownloadComplete -> OtaState.VERIFYING
        is OtaEvent.DownloadPaused -> OtaState.DOWNLOAD_PAUSED
        is OtaEvent.DownloadCancelled -> OtaState.IDLE
        is OtaEvent.DownloadFailed -> OtaState.FAILED
        is OtaEvent.VerifyStart -> OtaState.VERIFYING
        is OtaEvent.VerifyComplete -> OtaState.INSTALLING
        is OtaEvent.VerifyFailed -> OtaState.FAILED
        is OtaEvent.InstallStart -> OtaState.INSTALLING
        is OtaEvent.InstallProgress -> OtaState.INSTALLING
        is OtaEvent.InstallVerifying -> OtaState.INSTALL_VERIFYING
```

```
        is OtaEvent.InstallFinalizing ->
        OtaState.INSTALL_FINALIZING
        is OtaEvent.InstallNeedsReboot -> OtaState.REBOOTING
        is OtaEvent.InstallComplete -> OtaState.REBOOTING
        is OtaEvent.InstallEngineError -> OtaState.FAILED
        is OtaEvent.InstallCancelled -> OtaState.IDLE
        is OtaEvent.RebootStart -> OtaState.REBOOTING
        is OtaEvent.CommitStart -> OtaState.COMMITTING
        is OtaEvent.Succeeded -> OtaState.SUCCEEDED
        is OtaEvent.Failed -> OtaState.FAILED
    }
}
```

State Transition Diagram:



---

3. update_engine Integration (CRITICAL)

3.1 Binder IPC Interface

`update_engine` exposes its control interface via Android Binder IPC. The AIDL definitions are located in the Android source tree at `system/update_engine/binder_bindings/`. The Helix OTA client must bind to the `IUpdateEngine` interface to control the update process.

3.1.1 IUpdateEngine AIDL Interface

```
// IUpdateEngine.aidl
package android.os;

import android.os.IUpdateEngineCallback;

interface IUpdateEngine {
    /**
     * Applies the payload at the given URL. The payload must be
accessible
     * by the update_engine process (typically via file:// URI).
     *
     * @param url          The URL of the payload (e.g., file:///
data/ota/payload.bin)
     * @param offset       The offset within the file where the
payload starts
     * @param size         The size of the payload in bytes
     * @param headers      Key-value pairs from
payload_properties.txt
     */
    void applyPayload(String url, in long[] offset, in long[]
size, in String[] headers);

    /**
     * Suspends the currently active update.
     * The update can be resumed by calling resume().
     */
    void suspend();

    /**
     * Resumes a previously suspended update.
     */
    void resume();

    /**
     * Cancels the currently active update.
     * The device will remain on the current slot.
     */
    void cancel();

    /**
     * Registers a callback to receive status and completion
events.
     */
    void bind(IUpdateEngineCallback callback);

    /**
     * Unregisters a previously registered callback.
     */
    void unbind(IUpdateEngineCallback callback);
}
```

3.1.2 IUpdateEngineCallback AIDL Interface

```
// IUpdateEngineCallback.aidl
package android.os;

oneway interface IUpdateEngineCallback {
    /**
     * Called periodically with status updates during payload
application.
     *
     * @param status      One of UPDATE_STATUS_* constants
     * @param percentage  Progress percentage (0.0 to 1.0)
     */
    void onStatusUpdate(int status, float percentage);

    /**
     * Called when payload application completes (successfully or
with error).
     *
     * @param errorCode   One of UpdateEngineError constants.
     *                    SUCCESS (0) indicates successful
completion.
     */
    void onPayloadApplicationComplete(int errorCode);
}
```

3.2 UpdateEngineProxy — Binder Wrapper

The `UpdateEngineProxy` class encapsulates Binder IPC with `update_engine`, handling service binding, reconnection, and thread safety.

```
class UpdateEngineProxy(context: Context) {

    private val serviceManager by lazy {
        context.getSystemService(Context.UPDATE_ENGINE_SERVICE) }
    private var iUpdateEngine: IUpdateEngine? = null
    private var callback: IUpdateEngineCallback? = null
    private val connection = object : ServiceConnection {
        override fun onServiceConnected(name: ComponentName?,
        service: IBinder?) {
            iUpdateEngine =
        IUpdateEngine.Stub.asInterface(service)
            callback?.let { iUpdateEngine?.bind(it) }
        }
        override fun onServiceDisconnected(name: ComponentName?) {
            iUpdateEngine = null
        }
    }

    fun bind(callback: IUpdateEngineCallback) {
        this.callback = callback
        val intent = Intent("com.android.otatest.IUpdateEngine")
        intent.setPackage("com.android.otatest")
```

```
        val bound = context.bindService(
            intent, connection, Context.BIND_AUTO_CREATE
        )
        if (!bound) {
            Log.e("UpdateEngineProxy", "Failed to bind to
        update_engine service")
            throw UpdateEngineBindException("Cannot bind to
        update_engine")
        }
    }

    fun applyPayload(url: String, offset: LongArray, size:
        LongArray, headers: Array<String>) {
        iUpdateEngine?.applyPayload(url, offset, size, headers)
            ?: throw UpdateEngineNotBoundException()
    }

    fun suspend() { iUpdateEngine?.suspend() }
    fun resume() { iUpdateEngine?.resume() }
    fun cancel() { iUpdateEngine?.cancel() }
    fun unbind() {
        callback?.let { iUpdateEngine?.unbind(it) }
        context.unbindService(connection)
    }
}
```

3.3 Passing Payload Data to update_engine

The `applyPayload` call requires the following arguments:

Argument	Description	Example
`url`	File URI pointing to `payload.bin`	`file:///data/ota/update/payload.bin`
`offset`	Offset array (typically `[0]`)	`longArrayOf(0)`
`size`	Size array (typically `[0]` for entire file)	`longArrayOf(0)`
`headers`	Key-value pairs from `payload_properties.txt`	See below

The headers array must contain these key-value pairs, extracted from `payload_properties.txt`:

```
FILE_HASH=<sha256_hex_of_payload>
FILE_SIZE=<size_of_payload_in_bytes>
METADATA_HASH=<sha256_hex_of_manifest>
METADATA_SIZE=<size_of_manifest_in_bytes>
```

Example `applyPayload` invocation:

```
val payloadUrl = "file:///data/ota/update/payload.bin"
val offset = longArrayOf(0)
```

```
val size = longArrayOf(0)  // 0 means entire file
val headers = arrayOf(
    "FILE_HASH=2d9f3c1e5b7a8d4e6f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e",
    "FILE_SIZE=1234567890",
    "METADATA_HASH=a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b
    "METADATA_SIZE=98765"
)

updateEngine.applyPayload(payloadUrl, offset, size, headers)
```

3.4 Boot Control HAL Interaction

After `update_engine` completes payload application and reports `UPDATED_NEED_REBOOT`, the client must instruct the boot control HAL to set the active slot and initiate a reboot.

```
object BootControlHelper {

    private const val HAL_SERVICE_NAME =
        "android.hardware.boot.IBootControl/default"

    fun getCurrentSlot(): Int {
        val process = Runtime.getRuntime().exec(
            arrayOf("bootctl", "get-suffix")
        )
        val output =
        process.inputStream.bufferedReader().readText().trim()
        // Output is "_a" or "_b"; return 0 or 1
        return if (output.endsWith("_b")) 1 else 0
    }

    fun setActiveSlot(slot: Int) {
        // Via bootctl command-line tool (available on debug
        builds)
        // Or via HIDL/AIDL HAL interface
        Runtime.getRuntime().exec(
            arrayOf("bootctl", "set-active-boot-slot",
        slot.toString())
        )
    }

    fun markBootSuccessful() {
        Runtime.getRuntime().exec(
            arrayOf("bootctl", "mark-boot-successful")
        )
    }

    fun isSlotBootable(slot: Int): Boolean {
        val process = Runtime.getRuntime().exec(
            arrayOf("bootctl", "is-slot-bootable",
        slot.toString())
        )
        return process.waitFor() == 0
```

```
    }
}
```

3.5 Post-Install Verification Flow

After `update_engine` reports `UPDATED_NEED_REBOOT`:

1. **Client** calls `BootControlHelper.setActiveSlot(inactiveSlot)` — marks the newly updated slot as the boot target.

2. **Client** schedules a reboot via `PowerManager.reboot("ota")`.

3. **Bootloader** reads the `misc` partition to determine the active slot and boots from it.

4. **Android boots** on the new slot. The `BootCompletedReceiver` fires:

```
class BootCompletedReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_BOOT_COMPLETED) {
            // Verify we booted on the new slot
            val currentSlot =
        BootControlHelper.getCurrentSlot()
            val expectedSlot = prefs.targetSlot
            if (currentSlot == expectedSlot) {
                BootControlHelper.markBootSuccessful()
                stateMachine.transition(OtaEvent.CommitStart)
                // Trigger merge via update_engine
                val mergeIntent = Intent(context,
        MergeService::class.java)
                context.startForegroundService(mergeIntent)
            } else {
                // We rolled back; update failed

        stateMachine.transition(OtaEvent.Failed("Booted into
        wrong slot"))
            }
        }
    }
}
```

5. **Merge phase** — `update_engine` merges COW snapshots into base partitions. This can take several minutes on large partitions.

6. **After merge** — `BootControlHelper.markBootSuccessful()` finalizes the slot. The state machine transitions to `SUCCEEDED`.

---

4. A/B Partition Layout for RK3588

4.1 Complete Partition Table

The Orange Pi 5 Max (RK3588) uses a Virtual A/B partition layout. The following table shows all partitions required for Android 15 OTA support:

Partition	Size	A/B	Description
`boot_a` / `boot_b`	64 MB	Yes	Kernel + ramdisk (GKI boot image)
`init_boot_a` / `init_boot_b`	32 MB	Yes	Init ramdisk (Android 13+ GKI split)
`dtbo_a` / `dtbo_b`	16 MB	Yes	Device Tree Blob Overlay
`vbmeta_a` / `vbmeta_b`	4 MB	Yes	Verified Boot metadata
`vbmeta_system_a` / `vbmeta_system_b`	4 MB	Yes	System vbmeta chain partition
`vbmeta_vendor_a` / `vbmeta_vendor_b`	4 MB	Yes	Vendor vbmeta chain partition
`super`	8192 MB	Dynamic	Contains dynamic partitions (see below)
`userdata`	Remaining	No	User data (encrypted)
`misc`	4 MB	No	Boot control data, recovery intent
`metadata`	32 MB	No	Device-specific metadata
`persist`	16 MB	No	Persistent data (calibration, etc.)
`cache`	512 MB	No	OTA cache / recovery log
`resource`	8 MB	No	Rockchip resource (logo, etc.)

Dynamic partitions inside `super`:

Logical Partition	Typical Size	A/B	Description
`system_a` / `system_b`	3072 MB	Yes	Android system framework
`vendor_a` / `vendor_b`	1024 MB	Yes	Vendor HALs & libraries
`product_a` / `product_b`	512 MB	Yes	Product-specific apps & config
`odm_a` / `odm_b`	256 MB	Yes	ODM customizations
`system_ext_a` / `system_ext_b`	512 MB	Yes	System extensions

4.2 Parameter.txt Format for Rockchip

Rockchip uses a `parameter.txt` file to define the partition layout for the firmware image. This file is processed by Rockchip's `upgrade_tool` and the Android build system.

```
FIRMWARE_VER: 15.0.0
MACHINE_MODEL: Orange_Pi_5_Max
MACHINE_ID: RK3588
MANUFACTURER: Rockchip
MAGIC: 0x5041524B
ATAG: 0x00200000 0x00000400
CMDLINE: console=ttyS2,1500000n8
  androidboot.mode=normal
  androidboot.hardware=rk3588_orange_pi_5_max
  androidboot.console=ttyS2
  androidboot.verifiedbootstate=orange
  androidboot.slot_suffix=_a
  firmware_class.path=/vendor/etc/firmware
  init=/init
  loop.max_part=7
  buildvariant=userdebug

# Partition definitions: @<offset>(<name>, <size>, <type>)
# Type: 0x4 (bootable), 0x2 (read-only), 0x1 (write-only)
# Sizes in sectors (512 bytes per sector)

0x00002000@0x00004000(resource,0x00002000,0x2)
0x00004000@0x00006000(misc,0x00004000,0x0)
0x00002000@0x0000a000(dtbo_a,0x00002000,0x4)
0x00002000@0x0000c000(dtbo_b,0x00002000,0x4)
0x00080000@0x0000e000(boot_a,0x00080000,0x4)
0x00080000@0x0008e000(boot_b,0x00080000,0x4)
0x00040000@0x0010e000(init_boot_a,0x00040000,0x4)
0x00040000@0x0014e000(init_boot_b,0x00040000,0x4)
0x00002000@0x0018e000(vbmeta_a,0x00002000,0x4)
0x00002000@0x00190000(vbmeta_b,0x00002000,0x4)
0x00002000@0x00192000(vbmeta_system_a,0x00002000,0x2)
0x00002000@0x00194000(vbmeta_system_b,0x00002000,0x2)
0x00002000@0x00196000(vbmeta_vendor_a,0x00002000,0x2)
0x00002000@0x00198000(vbmeta_vendor_b,0x00002000,0x2)
0x10000000@0x0019a000(super,0x10000000,0x2)
0x00100000@0x1019a000(metadata,0x00100000,0x0)
0x00010000@0x1029a000(persist,0x00010000,0x0)
0x00200000@0x102aa000(cache,0x00200000,0x0)
-@0x104aa000(userdata)
```

4.3 U-Boot Configuration for A/B Boot

U-Boot on RK3588 must be configured to support A/B slot selection. The key configuration elements:

```
# defconfig additions for A/B support
CONFIG_ANDROID_AB=y
CONFIG_ANDROID_BOOT_IMAGE=y
CONFIG_ANDROID_BOOT_CONTROL=y
CONFIG_CMD_AB_SELECT=y
CONFIG_PARTITION_TYPE_GUID=y
CONFIG_CMD_GPT=y

# Boot script parameters
# U-Boot reads the 'misc' partition to determine the active slot
# The android_boot_control module implements:
#   - ab_select_slot(): Reads misc partition boot_ctrl struct
#   - ab_set_active(): Sets the active slot for next boot
#   - ab_mark_boot_successful(): Marks current boot as successful
```

The boot flow in U-Boot:

1. **Power-on** → U-Boot initializes hardware.
2. U-Boot calls `ab_select_slot()` which reads the `misc` partition's `boot_ctrl` struct.
3. The `boot_ctrl` struct (defined in `boot_control.h`) contains:

```
struct boot_ctrl {
    uint8_t  magic[4];          // "A/B" magic
    uint8_t  major_version;
    uint8_t  minor_version;
    uint8_t  nb_slot : 4;       // Number of slots (2 for A/B)
    uint8_t  reserved0 : 4;
    uint8_t  slot_info[2];      // Per-slot info (priority,
        tries, successful)
    uint8_t  recovery_tries_left;
    uint8_t  reserved1[57];
};
```

4. U-Boot selects the slot with highest priority that has `tries_remaining > 0` or `successful_boot == 1`.
5. U-Boot loads `boot_<suffix>` and `init_boot_<suffix>` from the selected slot.
6. If the boot fails 3 consecutive times (decrementing `tries_remaining`), the slot is deprioritized and the alternate slot is used.

4.4 Boot Control HAL Implementation for RK3588

Android 15 requires the AIDL Boot Control HAL (`android.hardware.boot`). For RK3588, this is implemented as a vendor HAL:

```
device/rockchip/rk3588/boot_control/
├── Android.bp
├── BootControl.cpp
```

```
├── BootControl.h
└── service.cpp

// BootControl.cpp — RK3588 AIDL Boot Control HAL
#include "BootControl.h"
#include <android-base/logging.h>
#include <android-base/properties.h>
#include <bootctrl/bootctrl.h>

namespace aidl::android::hardware::boot {

ndk::ScopedAStatus BootControl::getActiveBootSlot(int*
        _aidl_return) {
    // Read misc partition to determine active slot
    boot_ctrl ctrl;
    if (read_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(1);
    }
    for (int i = 0; i < ctrl.nb_slot; i++) {
        if (ctrl.slot_info[i].priority > 0 &&
            (ctrl.slot_info[i].successful_boot ||
        ctrl.slot_info[i].tries_remaining > 0)) {
            *_aidl_return = i;
            return ndk::ScopedAStatus::ok();
        }
    }
    *_aidl_return = 0;
    return ndk::ScopedAStatus::ok();
}

ndk::ScopedAStatus BootControl::markBootSuccessful() {
    boot_ctrl ctrl;
    if (read_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(1);
    }
    int current = getCurrentSlot();
    ctrl.slot_info[current].successful_boot = 1;
    ctrl.slot_info[current].tries_remaining = 7; // Reset tries
    if (write_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(2);
    }
    return ndk::ScopedAStatus::ok();
}

ndk::ScopedAStatus BootControl::setActiveBootSlot(int slot) {
    boot_ctrl ctrl;
    if (read_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(1);
    }
    // Lower priority of current active slot
    for (int i = 0; i < ctrl.nb_slot; i++) {
        if (i != slot) {
            ctrl.slot_info[i].priority = 0;
```

```
        }
    }
    // Set target slot as active
    ctrl.slot_info[slot].priority = 15; // Max priority
    ctrl.slot_info[slot].tries_remaining = 7;
    ctrl.slot_info[slot].successful_boot = 0;
    if (write_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(2);
    }
    return ndk::ScopedAStatus::ok();
}

ndk::ScopedAStatus BootControl::setSlotAsUnbootable(int slot) {
    boot_ctrl ctrl;
    if (read_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(1);
    }
    ctrl.slot_info[slot].priority = 0;
    ctrl.slot_info[slot].successful_boot = 0;
    ctrl.slot_info[slot].tries_remaining = 0;
    if (write_boot_ctrl(&ctrl) != 0) {
        return ndk::ScopedAStatus::fromServiceSpecificError(2);
    }
    return ndk::ScopedAStatus::ok();
}

// ... remaining AIDL methods: getNumberSlots, getCurrentSlot,
       getSuffix,
//     isSlotBootable, isSlotMarkedSuccessful

}  // namespace aidl::android::hardware::boot
```

---

5. OTA Zip Format Handling

5.1 OTA Zip Structure

A full OTA zip for Android 15 Virtual A/B has the following structure:

```
helix_ota_rk3588_1.1.0.zip
├── payload.bin                 # Main OTA payload
(DeltaGenerator output)
├── payload_properties.txt      # Metadata for update_engine
├── care_map.pb                 # Care map for dm-verity
(protobuf)
├── META-INF/
│   └── com/
│       ├── android/
│       │   ├── metadata        # Build metadata (fingerprint,
device, etc.)
│       │   └── otacert         # OTA signing certificate (PEM)
```

```
│       └── google/
│           └── android/
│               ├── update-binary   # Not used in A/B (legacy)
│               └── updater-binary  # Not used in A/B (legacy)
└── apex/                         # APEX updates (if any)
    └── com.android.runtime/
        └── com.android.runtime.apex
```

5.2 payload.bin Structure

The `payload.bin` file is a binary format defined by Chrome OS's `update_engine` (which Android adopted). Its structure:

```
┌──────────────────────────────────────────────────────────┐
│ CrAU (Cr50 AU) Header                                    │
│   magic[4]:        "CrAU"                                │
│   major_version:   uint64  (currently 1 or 2)            │
│   manifest_size:   uint64  (size of protobuf manifest)   │
│   metadata_sig_size: uint32 (only if major_version >= 2) │
│   metadata_signature: bytes  (RSA signature over manifest)│
├──────────────────────────────────────────────────────────┤
│ Manifest (protobuf, size = manifest_size)                │
│   AcquireLockId:    repeated string                      │
│   ReleaseLockId:    repeated string                      │
│   install_operations:     repeated InstallOperation      │
│   kernel_install_operations: repeated InstallOperation   │
│   partitions:       repeated PartitionUpdate             │
│     ├── partition_name:  string (e.g., "system")         │
│     ├── old_partition_info: PartitionInfo (hash, size)   │
│     ├── new_partition_info: PartitionInfo (hash, size)   │
│     ├── operations: repeated InstallOperation            │
│     │     ├── type:     enum (REPLACE, REPLACE_BZ,       │
│     │     │               REPLACE_XZ, ZERO, SOURCE_COPY, │
│     │     │               SOURCE_BROTLI, LZ4IF)          │
│     │     ├── data_offset: uint64                        │
│     │     ├── data_length: uint64                        │
│     │     ├── src_extents:  repeated Extent              │
│     │     └── dst_extents:  repeated Extent              │
│     ├── estimate_cow_size: uint64                        │
│     └── postinstall:    PostInstallConfig                │
│   max_timestamp:    int64                                │
│   dynamic_partition_metadata: DynamicPartitionMetadata   │
│   partial_update:   bool                                 │
├──────────────────────────────────────────────────────────┤
│ Extra Data Block                                         │
│   [Binary data for REPLACE/REPLACE_BZ/etc. operations]   │
│   (referenced by data_offset and data_length in operations)│
└──────────────────────────────────────────────────────────┘
```

The client does **not** parse `payload.bin` directly — that is `update_engine`'s responsibility. However, the client must:

1. Know the payload's total size for download progress tracking.

2. Know the `metadata_size` (from `payload_properties.txt`) to verify the manifest hash.
3. Ensure the file is accessible to the `update_engine` process.

5.3 payload_properties.txt Format and Parsing

This file is generated by the build system's `delta_generator` tool alongside `payload.bin`. It contains the metadata needed by `update_engine` to locate and validate the payload:

```
FILE_HASH=2d9f3c1e5b7a8d4e6f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9
FILE_SIZE=1234567890
METADATA_HASH=a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c
METADATA_SIZE=98765
```

Field	Description
`FILE_HASH`	SHA-256 hash of the entire `payload.bin` file (hex-encoded)
`FILE_SIZE`	Total size of `payload.bin` in bytes
`METADATA_HASH`	SHA-256 hash of the manifest portion of the payload
`METADATA_SIZE`	Size of the manifest (protobuf) in bytes

Parsing in Kotlin:

```
data class PayloadProperties(
    val fileHash: String,
    val fileSize: Long,
    val metadataHash: String,
    val metadataSize: Long
) {
    fun toHeadersArray(): Array<String> = arrayOf(
        "FILE_HASH=$fileHash",
        "FILE_SIZE=$fileSize",
        "METADATA_HASH=$metadataHash",
        "METADATA_SIZE=$metadataSize"
    )

    companion object {
        fun parse(file: File): PayloadProperties {
            val lines = file.readLines()
                .filter { it.contains("=") }
                .associate {
                    val parts = it.split("=", limit = 2)
                    parts[0].trim() to parts[1].trim()
                }

            return PayloadProperties(
                fileHash = lines["FILE_HASH"]
                    ?: throw IllegalArgumentException("Missing
        FILE_HASH"),
```

```
                fileSize = lines["FILE_SIZE"]?.toLongOrNull()
                    ?: throw IllegalArgumentException("Missing or
        invalid FILE_SIZE"),
                metadataHash = lines["METADATA_HASH"]
                    ?: throw IllegalArgumentException("Missing
        METADATA_HASH"),
                metadataSize =
        lines["METADATA_SIZE"]?.toLongOrNull()
                    ?: throw IllegalArgumentException("Missing or
        invalid METADATA_SIZE")
            )
        }
    }
}
```

5.4 care_map.pb Purpose and Processing

The `care_map.pb` file is a protobuf-encoded care map that tells `dm-verity` which blocks of the updated partitions contain meaningful data (as opposed to zero-filled blocks). This is critical for Virtual A/B because:

- During the merge phase, only the blocks listed in the care map need to be copied from the COW file to the base partition.
- This dramatically reduces merge time for sparse partitions (e.g., a 4 GB system partition with only 2 GB of actual data).

The care map protobuf schema:

```
message CareMap {
    message Entry {
        required string id =
        1;           // Partition name (e.g., "system_b")
        repeated Range ranges = 2;        // Block ranges with
        data
    }
    message Range {
        required int64 start = 1;         // Start block
        required int64 end = 2;           // End block (exclusive)
    }
    repeated Entry entries = 1;
}
```

The client extracts `care_map.pb` from the OTA zip and writes it to `/data/ota/care_map.pb`. The `update_engine` daemon reads this file during the merge phase.

5.5 Hash File Verification

The Helix OTA build pipeline generates supplementary hash files alongside the OTA zip:

- `helix_ota_rk3588_1.1.0.zip.sha256` — SHA-256 hash of the complete OTA zip

- `helix_ota_rk3588_1.1.0.zip.sha256.sig` — RSA-2048 signature of the SHA-256 hash

The `.sha256` file format:

```
2d9f3c1e5b7a8d4e6f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e
helix_ota_rk3588_1.1.0.zip
```

The client downloads both the OTA zip and the `.sha256` file, verifying the zip against the hash before proceeding.

---

6. Download Manager

6.1 Chunked Download with Resume Support

The `ChunkedDownloader` implements HTTP Range-based downloading with chunk verification and resume capability. If a download is interrupted (network loss, user pause, app crash), it resumes from the last completed chunk boundary.

```
class ChunkedDownloader(
    private val httpClient: OkHttpClient,
    private val chunkSize: Long = 4 * 1024 * 1024, // 4 MB chunks
    private val bandwidthThrottle: BandwidthThrottle =
        BandwidthThrottle.UNLIMITED
) {
    private val scope = CoroutineScope(Dispatchers.IO +
        SupervisorJob())
    private var activeJob: Job? = null
    private var isPaused = false

    fun download(
        url: String,
        targetFile: File,
        headers: Map<String, String>,
        listener: DownloadProgressListener
    ) {
        activeJob = scope.launch {
            val stateFile = File(targetFile.parent, "$
        {targetFile.name}.state")

            // Determine starting position from state file
            val startPos = if (stateFile.exists() &&
        targetFile.exists()) {
                val state =
        Json.decodeFromString<DownloadState>(stateFile.readText())
                state.completedBytes
            } else {
                0L
            }

            // Get total size via HEAD request
```

```
    val totalSize = fetchContentLength(url, headers)
    if (startPos >= totalSize) {
        listener.onComplete(targetFile)
        return@launch
    }

    // Disk space check
    val requiredSpace = totalSize - startPos + (512L *
1024 * 1024) // 512 MB buffer
    if (targetFile.usableSpace < requiredSpace) {
        listener.onError(DownloadError.INSUFFICIENT_SPACE)
        return@launch
    }

    var downloaded = startPos
    val buffer = ByteArray(8192)
    val throttledStream = bandwidthThrottle.wrap(
        // Stream will be created per-chunk below
        NullInputStream()
    )

    var currentPos = startPos
    while (currentPos < totalSize && isActive) {
        if (isPaused) {
            saveState(stateFile, currentPos, totalSize)
            break
        }

        val chunkEnd = minOf(currentPos + chunkSize,
totalSize)
        val rangeHeader = "bytes=$currentPos-$chunkEnd"

        val request = Request.Builder()
            .url(url)
            .apply { headers.forEach { (k, v) ->
addHeader(k, v) } }
            .addHeader("Range", rangeHeader)
            .build()

        try {
            val response =
httpClient.newCall(request).execute()
            if (!response.isSuccessful && response.code !
= 206) {

listener.onError(DownloadError.HTTP_ERROR(response.code))
                return@launch
            }

            response.body?.byteStream()?.use { stream ->
                val throttled =
bandwidthThrottle.wrap(stream)
                var lastReportTime =
System.currentTimeMillis()
```

```
                    var bytesInChunk = 0L

                    while (bytesInChunk < (chunkEnd -
    currentPos)) {
                        val read = throttled.read(buffer)
                        if (read == -1) break

                        // Append to file
                        targetFile.appendBytes(buffer, 0,
    read)
                        bytesInChunk += read
                        downloaded += read

                        // Report progress (throttled to 500ms
    intervals)
                        val now = System.currentTimeMillis()
                        if (now - lastReportTime > 500) {
                            listener.onProgress(downloaded,
    totalSize,

    bandwidthThrottle.currentRateBps)
                                lastReportTime = now
                        }
                    }
                }

                currentPos = chunkEnd
                saveState(stateFile, currentPos, totalSize)

            } catch (e: IOException) {
                saveState(stateFile, currentPos, totalSize)

    listener.onError(DownloadError.NETWORK_ERROR(e.message ?:
    "IO error"))
                return@launch
            }
        }

        if (downloaded >= totalSize) {
            stateFile.delete()
            listener.onComplete(targetFile)
        }
    }
}

private suspend fun fetchContentLength(
    url: String,
    headers: Map<String, String>
): Long {
    return withContext(Dispatchers.IO) {
        val request = Request.Builder()
            .url(url)
            .apply { headers.forEach { (k, v) -> addHeader(k,
    v) } }
```

```
                .head()
                .build()
            val response = httpClient.newCall(request).execute()
            response.body?.contentLength()
                ?: throw IOException("Cannot determine content
        length")
        }
    }

    private fun saveState(stateFile: File, completedBytes: Long,
        totalBytes: Long) {
        stateFile.writeText(Json.encodeToString(
            DownloadState(completedBytes, totalBytes)
        ))
    }

    fun pause() { isPaused = true }
    fun cancel() { activeJob?.cancel(); isPaused = false }

    data class DownloadState(val completedBytes: Long, val
        totalBytes: Long)
}

interface DownloadProgressListener {
    fun onProgress(downloaded: Long, total: Long, speedBps: Long)
    fun onComplete(file: File)
    fun onError(error: DownloadError)
}
```

6.2 Bandwidth Throttling

```
class BandwidthThrottle(private val maxBytesPerSecond: Long) {

    companion object {
        val UNLIMITED = BandwidthThrottle(Long.MAX_VALUE)
        val WIFI_DEFAULT = BandwidthThrottle(10 * 1024 * 1024) //
        10 MB/s
        val CELLULAR_DEFAULT = BandwidthThrottle(2 * 1024 *
        1024) // 2 MB/s
    }

    var currentRateBps: Long = 0
        private set

    private var tokens = maxBytesPerSecond
    private var lastRefillTime = System.nanoTime()

    @Synchronized
    fun wrap(inputStream: InputStream): ThrottledInputStream {
        return ThrottledInputStream(inputStream, this)
    }

    @Synchronized
```

```
    fun acquire(bytes: Int): Long {
        if (maxBytesPerSecond == Long.MAX_VALUE) return 0L

        val now = System.nanoTime()
        val elapsedNanos = now - lastRefillTime
        tokens = minOf(maxBytesPerSecond,
            tokens + (elapsedNanos * maxBytesPerSecond /
        1_000_000_000))
        lastRefillTime = now

        return if (tokens >= bytes) {
            tokens -= bytes
            0L
        } else {
            val deficit = bytes - tokens
            tokens = 0
            val waitMs = (deficit * 1000 / maxBytesPerSecond)
            Thread.sleep(waitMs)
            waitMs
        }
    }
}

class ThrottledInputStream(
    private val delegate: InputStream,
    private val throttle: BandwidthThrottle
) : InputStream() {
    override fun read(): Int {
        throttle.acquire(1)
        return delegate.read()
    }
    override fun read(b: ByteArray, off: Int, len: Int): Int {
        throttle.acquire(len)
        val read = delegate.read(b, off, len)
        throttle.currentRateBps = len.toLong() // Simplified
        return read
    }
    override fun close() = delegate.close()
}
```

6.3 Network Policy

```
class NetworkPolicyManager(context: Context) {

    private val connectivityManager =
        context.getSystemService(Context.CONNECTIVITY_SERVICE) as
        ConnectivityManager

    enum class NetworkPolicy {
        WIFI_ONLY,
        WIFI_PREFERRED,    // WiFi preferred, cellular with user
        consent
        ANY_NETWORK
```

```
    }

    fun shouldDownload(policy: NetworkPolicy): Boolean {
        val network = connectivityManager.activeNetwork ?: return
        false
        val caps =
        connectivityManager.getNetworkCapabilities(network) ?:
        return false

        return when (policy) {
            NetworkPolicy.WIFI_ONLY ->

        caps.hasTransport(NetworkCapabilities.TRANSPORT_WIFI)
            NetworkPolicy.WIFI_PREFERRED ->

        caps.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) ||

        caps.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR) //
        Consent handled at UI level
            NetworkPolicy.ANY_NETWORK ->

        caps.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) ||

        caps.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR)
        ||

        caps.hasTransport(NetworkCapabilities.TRANSPORT_ETHERNET)
        }
    }

    fun isOnWifi(): Boolean {
        val network = connectivityManager.activeNetwork ?: return
        false
        val caps =
        connectivityManager.getNetworkCapabilities(network) ?:
        return false
        return
        caps.hasTransport(NetworkCapabilities.TRANSPORT_WIFI)
    }

    fun isOnEthernet(): Boolean {
        val network = connectivityManager.activeNetwork ?: return
        false
        val caps =
        connectivityManager.getNetworkCapabilities(network) ?:
        return false
        return
        caps.hasTransport(NetworkCapabilities.TRANSPORT_ETHERNET)
    }
}
```

6.4 Disk Space Verification

```
class DownloadStorageManager(context: Context) {

    private val otaDir = File(context.filesDir, "ota").also {
        it.mkdirs() }

    fun getOtaZipFile(): File = File(otaDir, "update.zip")
    fun getExtractDir(): File = File(otaDir, "extracted").also {
        it.mkdirs() }

    fun hasSufficientSpace(requiredBytes: Long): Boolean {
        val buffer = 512L * 1024 * 1024 // 512 MB buffer for COW
        and temp files
        val available = otaDir.usableSpace
        return available >= (requiredBytes + buffer)
    }

    fun getRequiredSpace(updateSize: Long): Long {
        // OTA zip + extracted payload + COW estimate
        return updateSize + (updateSize * 1.5).toLong()
    }

    fun cleanup() {
        otaDir.listFiles()?.forEach { file ->
            if (file.isFile) file.delete()
        }
        getExtractDir().deleteRecursively()
    }

    fun deletePartialFiles() {
        otaDir.listFiles()?.filter {
            it.name.endsWith(".state") ||
        it.name.endsWith(".part")
        }?.forEach { it.delete() }
    }
}
```

---

7. Verification Engine

The verification pipeline consists of five sequential steps. Each step must pass before proceeding to the next. If any step fails, the process is aborted with a specific error code.

7.1 Step 1: ZIP Structural Integrity

```
class ZipVerifier {

    fun verify(file: File): ZipVerificationResult {
        try {
```

```
            ZipFile(file).use { zip ->
                // Verify central directory is readable
                val entries = zip.entries()
                val requiredFiles = listOf(
                    "payload.bin",
                    "payload_properties.txt"
                )
                val foundFiles = mutableSetOf<String>()
                while (entries.hasMoreElements()) {
                    foundFiles.add(entries.nextElement().name)
                }

                val missing = requiredFiles.filter { it !in
        foundFiles }
                if (missing.isNotEmpty()) {
                    return ZipVerificationResult.Invalid(
                        "Missing required files: $
        {missing.joinToString()}"
                    )
                }

                // Verify payload.bin is non-empty
                val payloadEntry = zip.getEntry("payload.bin")
                if (payloadEntry.size <= 0) {
                    return
        ZipVerificationResult.Invalid("payload.bin is empty or
        corrupt")
                }

                return ZipVerificationResult.Valid(
                    payloadSize = payloadEntry.size,
                    hasCareMap =
        foundFiles.contains("care_map.pb"),
                    hasOtacert = foundFiles.contains("META-INF/
        com/android/otacert")
                )
            }
        } catch (e: ZipException) {
            return ZipVerificationResult.Invalid("ZIP corrupt: $
        {e.message}")
        } catch (e: IOException) {
            return ZipVerificationResult.Invalid("IO error: $
        {e.message}")
        }
    }
}

sealed class ZipVerificationResult {
    data class Valid(
        val payloadSize: Long,
        val hasCareMap: Boolean,
        val hasOtacert: Boolean
    ) : ZipVerificationResult()
```

```
    data class Invalid(val reason: String) :
        ZipVerificationResult()
    val isValid: Boolean get() = this is Valid
}
```

7.2 Step 2: SHA-256 Hash Verification

```
class HashVerifier {

    fun verify(file: File, expectedHash: String, hashFile: File?
        = null): Boolean {
        // If hash file is available, read expected hash from it
        val hash = hashFile?.let { readHashFromFile(it) } ?:
        expectedHash

        if (hash.isEmpty()) {
            Log.w("HashVerifier", "No hash available for
        verification")
            return false
        }

        val computedHash = computeSha256(file)
        val matches = computedHash.equals(hash, ignoreCase = true)

        if (!matches) {
            Log.e("HashVerifier",
                "Hash mismatch! Expected: $hash, Computed:
        $computedHash")
        }
        return matches
    }

    private fun readHashFromFile(hashFile: File): String {
        // Format: "<hash>  <filename>" (standard sha256sum
        output)
        val line = hashFile.readLines().firstOrNull() ?: return ""
        return line.split(" ").first().trim()
    }

    private fun computeSha256(file: File): String {
        val digest = MessageDigest.getInstance("SHA-256")
        file.inputStream().use { stream ->
            val buffer = ByteArray(8192)
            var read: Int
            while (stream.read(buffer).also { read = it } != -1) {
                digest.update(buffer, 0, read)
            }
        }
        return digest.digest().joinToString("") {
        "%02x".format(it) }
    }
}
```

7.3 Step 3: RSA Signature Verification

```
class SignatureVerifier(private val context: Context) {

    /**
     * Verifies the RSA signature on the payload manifest.
     * The OTA zip contains an otacert (signing certificate) in
     * META-INF/com/android/otacert. We verify that this
        certificate
     * matches one of the trusted certificates installed on the
        device,
     * then verify the signature in payload.bin against that
        certificate.
     */
    fun verifyPayloadManifest(otaZip: File, publicKey:
        PublicKey): Boolean {
        try {
            ZipFile(otaZip).use { zip ->
                // Read the OTA certificate from the zip
                val certEntry = zip.getEntry("META-INF/com/
        android/otacert")
                    ?: return false
                val certBytes =
        zip.getInputStream(certEntry).readBytes()
                val cert = CertificateFactory.getInstance("X.509")
                    .generateCertificate(certBytes.inputStream())
        as X509Certificate

                // Verify certificate is trusted (matches system
        OTA cert)
                if (!isTrustedCertificate(cert)) {
                    Log.e("SignatureVerifier", "OTA certificate
        not trusted")
                    return false
                }

                // Extract manifest from payload.bin
                val payloadEntry = zip.getEntry("payload.bin") ?:
        return false
                val payloadStream =
        zip.getInputStream(payloadEntry)

                // Read CrAU header
                val magic = ByteArray(4)
                payloadStream.read(magic)
                if (String(magic) != "CrAU") return false

                // Read major version
                val majorVersion =
        payloadStream.readLongBigEndian()
                // Read manifest size
                val manifestSize =
        payloadStream.readLongBigEndian()
```

```
            if (majorVersion >= 2) {
                val metadataSigSize =
    payloadStream.readIntBigEndian()
                // Read manifest
                val manifestBytes =
    ByteArray(manifestSize.toInt())
                payloadStream.read(manifestBytes)

                // Read metadata signature
                val sigBytes = ByteArray(metadataSigSize)
                payloadStream.read(sigBytes)

                // Verify signature
                val sig =
    Signature.getInstance("SHA256withRSA")
                sig.initVerify(publicKey)
                sig.update(manifestBytes)
                return sig.verify(sigBytes)
            }
        }
    } catch (e: Exception) {
        Log.e("SignatureVerifier", "Signature verification
    failed", e)
    }
    return false
}

private fun isTrustedCertificate(cert: X509Certificate):
    Boolean {
    // Compare with certificate stored in /system/etc/
    security/otacerts.zip
    val trustedCerts = loadTrustedOtaCerts()
    return trustedCerts.any { trusted ->
        cert.encoded.contentEquals(trusted.encoded)
    }
}

private fun loadTrustedOtaCerts(): List<X509Certificate> {
    val certs = mutableListOf<X509Certificate>()
    val otacertsZip = File("/system/etc/security/
    otacerts.zip")
    if (otacertsZip.exists()) {
        ZipFile(otacertsZip).use { zip ->
            val entries = zip.entries()
            while (entries.hasMoreElements()) {
                val entry = entries.nextElement()
                if (entry.name.endsWith(".x509.pem") ||
    entry.name.endsWith(".pem")) {
                    val cert =
    CertificateFactory.getInstance("X.509")
                        .generateCertificate(zip.getInputStream(entry))
    as X509Certificate
                    certs.add(cert)
                }
```

```
                }
            }
        }
        return certs
    }
}
```

7.4 Step 4: payload_properties.txt Format Validation

```
fun verifyPayloadProperties(otaZip: File): Boolean {
    try {
        ZipFile(otaZip).use { zip ->
            val entry = zip.getEntry("payload_properties.txt") ?:
        return false
            val content =
        zip.getInputStream(entry).bufferedReader().readText()

            val requiredKeys = setOf(
                "FILE_HASH", "FILE_SIZE", "METADATA_HASH",
        "METADATA_SIZE"
            )
            val presentKeys = mutableSetOf<String>()

            for (line in content.lines()) {
                if (!line.contains("=")) continue
                val key = line.split("=")[0].trim()
                presentKeys.add(key)

                // Validate value format
                val value = line.split("=", limit = 2)[1].trim()
                when (key) {
                    "FILE_HASH", "METADATA_HASH" -> {
                        if (!value.matches(Regex("^[a-fA-F0-9]{64}
        $"))) {
                            Log.e("Verify", "Invalid hash format
        for $key")
                            return false
                        }
                    }
                    "FILE_SIZE", "METADATA_SIZE" -> {
                        value.toLongOrNull() ?: run {
                            Log.e("Verify", "Invalid size format
        for $key")
                            return false
                        }
                    }
                }
            }

            if (presentKeys != requiredKeys) {
                Log.e("Verify", "Missing keys: ${(requiredKeys -
        presentKeys).joinToString()}")
                return false
```

```
            }
            return true
        }
    } catch (e: Exception) {
        return false
    }
}
```

7.5 Step 5: Device Compatibility Check

```
class DeviceCompatibilityCheck {

    fun verify(
        targetHardware: String,
        minVersion: String?,
        targetVersion: String
    ): Boolean {
        // Check hardware model
        val currentHardware = Build.HARDWARE // "rk3588"
        if (targetHardware != currentHardware) {
            Log.e("Compat", "Hardware mismatch:
        expected=$targetHardware, actual=$currentHardware")
            return false
        }

        // Check minimum current version
        minVersion?.let {
            val currentVersion = Build.DISPLAY
            if (compareVersions(currentVersion, it) < 0) {
                Log.e("Compat", "Current version too old:
        $currentVersion < $it")
                return false
            }
        }

        // Prevent downgrade (unless explicitly allowed by server)
        val currentVersion = Build.DISPLAY
        if (compareVersions(targetVersion, currentVersion) <= 0) {
            Log.w("Compat", "Target version not newer:
        $targetVersion <= $currentVersion")
            // Not a hard failure — could be a patch release
        }

        // Check battery level (must be > 30% for installation)
        // This is advisory for the check phase; enforced during
        install

        return true
    }

    private fun compareVersions(v1: String, v2: String): Int {
        val parts1 = v1.split(".")
        val parts2 = v2.split(".")
```

```
            for (i in 0 until maxOf(parts1.size, parts2.size)) {
                val p1 = parts1.getOrElse(i) { "0" }.removeSuffix("-
            mvp").toIntOrNull() ?: 0
                val p2 = parts2.getOrElse(i) { "0" }.removeSuffix("-
            mvp").toIntOrNull() ?: 0
                if (p1 != p2) return p1.compareTo(p2)
            }
            return 0
        }
}
```

7.6 Error Codes

```
enum class VerifyError(val code: Int, val description: String) {
    MISSING_UPDATE_INFO(1001, "No cached update information
        available"),
    ZIP_CORRUPT(1002, "OTA ZIP file is corrupt or incomplete"),
    HASH_MISMATCH(1003, "SHA-256 hash does not match expected
        value"),
    SIGNATURE_INVALID(1004, "RSA signature verification failed"),
    PROPERTIES_INVALID(1005,
        "payload_properties.txt is missing or invalid"),
    DEVICE_INCOMPATIBLE(1006, "Device hardware or version is
        incompatible"),
    VERIFICATION_EXCEPTION(1007, "Unexpected error during
        verification");

    companion object {
        fun fromCode(code: Int) = entries.find { it.code == code }
            ?: VERIFICATION_EXCEPTION
    }
}
```

---

8. Build Integration

8.1 Build Pipeline OTA Zip Production

The Helix OTA build pipeline extends the standard AOSP build process to produce OTA artifacts:

```
# Step 1: Build the system images
source build/envsetup.sh
lunch rk3588_orange_pi_5_max-userdebug
make -j$(nproc) target-files-package

# Step 2: Generate the full OTA package
# This runs ota_from_target_files which invokes delta_generator
python system/build/tools/releasetools/ota_from_target_files \
    -k build/target/product/security/testkey \
    -i previous_target_files.zip \
```

```
    out/target/product/rk3588_orange_pi_5_max/obj/PACKAGING/
        target_files_intermediates/rk3588_orange_pi_5_max-
        target_files.zip \
    helix_ota_rk3588_1.1.0.zip

# Step 3: Generate hash files
sha256sum helix_ota_rk3588_1.1.0.zip >
        helix_ota_rk3588_1.1.0.zip.sha256

# Step 4: Sign the hash file with the OTA release key
openssl dgst -sha256 -sign helix_ota_release_key.pem \
    -out helix_ota_rk3588_1.1.0.zip.sha256.sig \
    helix_ota_rk3588_1.1.0.zip.sha256

# Step 5: Upload all artifacts to the Helix OTA server
helix-cli upload \
    --file helix_ota_rk3588_1.1.0.zip \
    --hash-file helix_ota_rk3588_1.1.0.zip.sha256 \
    --sig-file helix_ota_rk3588_1.1.0.zip.sha256.sig \
    --target-hardware rk3588 \
    --target-version 1.1.0 \
    --min-version 1.0.0 \
    --changelog "Security patch March 2026, kernel 6.6 LTS update"
```

8.2 Signing Key Management

The OTA signing infrastructure uses a three-tier key system:

Key	Purpose	Storage
`helix_ota_release_key`	Signs the payload manifest (embedded in payload.bin)	HSM / secure build server
`helix_ota_hash_signing_key`	Signs the `.sha256` hash file	HSM / secure build server
`platform_key`	Signs the HelixOtaClient APK (must match system signature)	Build server keystore

Key rotation procedure:

1. Generate a new signing key pair.
2. Build an OTA zip signed with the **new** key.
3. The OTA zip includes both the old and new certificates in `META-INF/com/android/otacert`.
4. The `update_engine` accepts payloads signed by either certificate during the transition period.
5. After the first successful boot with the new key, subsequent OTAs use only the new key.

8.3 Device Tree / build.prop Configuration

The OTA server URL and client configuration are embedded in the device's build properties:

```
# build.prop additions for Helix OTA
ro.helix.ota.server_url=https://ota.helix.local/api/v1
ro.helix.ota.check_interval_hours=4
ro.helix.ota.device_model=rk3588_orange_pi_5_max
ro.helix.ota.network_policy=wifi_preferred
ro.helix.ota.max_download_bandwidth_bps=10485760
ro.helix.ota.min_battery_percent=30
ro.helix.ota.required_free_space_mb=512
```

These properties can be overridden via a `persistent.helix.ota.*` property for testing or enterprise configuration:

```
# Override for staging environment
persist.helix.ota.server_url=https://ota-staging.helix.local/api/
v1
persist.helix.ota.check_interval_hours=1
```

8.4 System App Integration

The `HelixOtaClient` APK is pre-installed as a privileged system app:

```
device/rockchip/rk3588/
├── HelixOtaClient/
│   ├── HelixOtaClient.apk    # Pre-built APK
│   └── privapp-permissions.xml
└── mkcombinedroot/
    └── ... (existing Rockchip build files)
```

privapp-permissions.xml — grants the app system-level permissions:

```
<?xml version="1.0" encoding="utf-8"?>
<permissions>
    <privapp-permissions package="com.helix.ota.client">
        <permission
        name="android.permission.INTERACT_ACROSS_USERS"/>
        <permission name="android.permission.REBOOT"/>
        <permission name="android.permission.RECOVERY"/>
        <permission name="android.permission.FOREGROUND_SERVICE"/>
        <permission
        name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE"/
        >
        <permission
        name="android.permission.UPDATE_DEVICE_STATISTICS"/>
        <permission
        name="android.permission.WRITE_SECURE_SETTINGS"/>
        <permission
        name="android.permission.CONNECTIVITY_INTERNAL"/>
    </privapp-permissions>
</permissions>
```

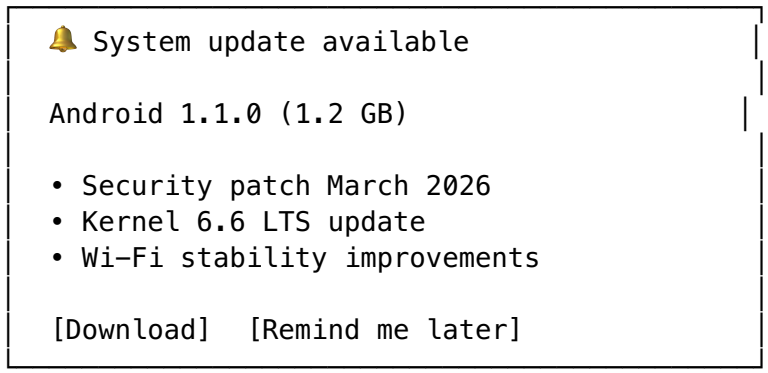
Android.bp entry for pre-built APK:

```
android_app_import {
    name: "HelixOtaClient",
    apk: "HelixOtaClient.apk",
    certificate: "platform",
    privileged: true,
    dex_preopt: {
        enabled: true,
    },
    required: ["privapp-permissions-HelixOtaClient.xml"],
}
```

---

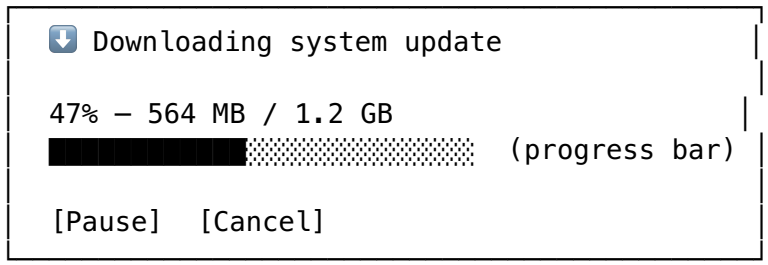
9. User Experience

9.1 Notification Flow for Update Availability



When the user taps **Download**: - The notification transitions to a download progress notification. - If the user taps **Remind me later**, a follow-up notification is scheduled for the next day (configurable, max 3 deferrals before auto-downloading on WiFi).

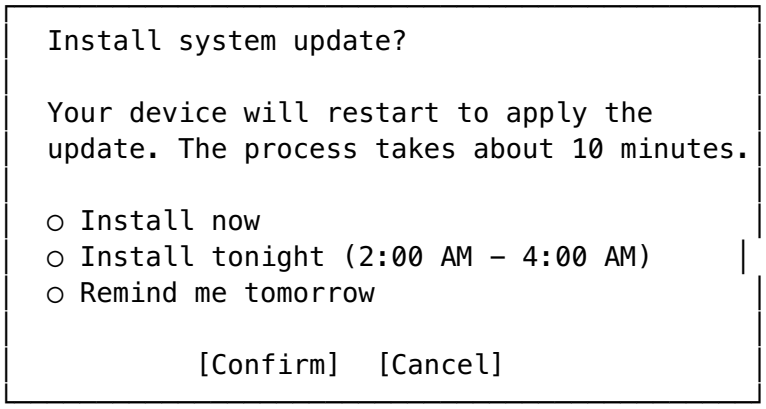
9.2 Download Progress Notification



The progress bar updates every 500ms. Speed and estimated time remaining are shown in the expanded notification.

9.3 Installation Scheduling

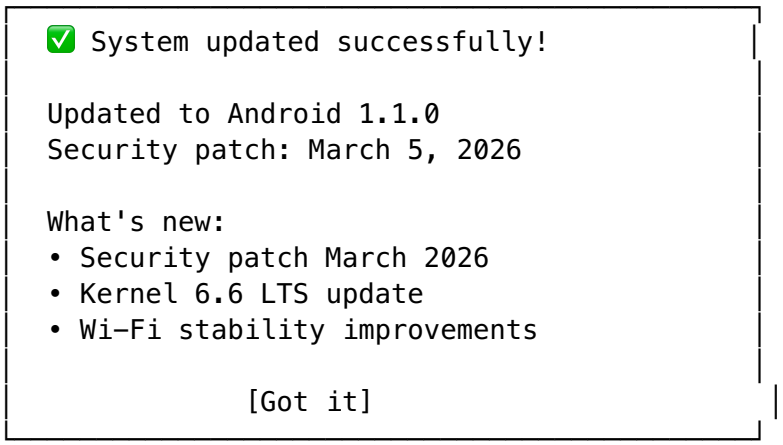
After verification, the user is presented with an installation scheduling dialog:



If “Install tonight” is selected, an `AlarmManager` exact alarm is set for the chosen time window. The device must be charging and have sufficient battery at the scheduled time.

9.4 Post-Update Welcome Screen

After a successful update and reboot, a welcome dialog is shown:



This is triggered by the `BootCompletedReceiver` detecting a version change between the last recorded version and the current `Build.DISPLAY`.

9.5 Error Handling and User Messaging

Error	User Message	Action
Network failure during download	“Download interrupted. Will retry when connected.”	Auto-retry on network restore
Insufficient disk space	“Not enough storage to download update. Free at least 2 GB.”	Manual cleanup required
Hash mismatch	“Update file is corrupted. Downloading again…”	Auto-retry download

Error	User Message	Action
Signature verification failed	“Update signature could not be verified. Contact your administrator.”	Critical — no retry
`update_engine` error	“Installation failed (error code: XX). The device is safe.”	Report to server, offer retry
Boot into wrong slot (rollback)	“Update could not be applied. Your system has been restored.”	Report to server, offer retry
Battery too low for install	“Battery too low to install. Charge to at least 30%.”	Wait for charging

10. Go Client SDK (Server-Communication Layer)

10.1 Architecture Overview

The Go Client SDK provides the server-communication layer shared between the Android client (via `gomobile` bind) and potential future platforms. It is compiled into an AAR via `gomobile bind` and called from Kotlin through generated bindings.

```
helix-ota-sdk-go/
├── go.mod
├── go.sum
├── client.go          # Main client entry point
├── check.go           # Update check API
├── download.go        # Download manager with resume
├── report.go          # Status reporting
├── tls.go             # mTLS certificate management
├── config.go          # Configuration types
├── models.go          # Shared data models
└── errors.go          # Error types
```

10.2 Client Initialization

```go
// client.go
package helixotasdk

import (
    "context"
    "crypto/tls"
    "crypto/x509"
    "fmt"
    "net/http"
    "os"
    "sync"
    "time"
)
```

```
// Client is the main SDK entry point for OTA operations.
type Client struct {
    config     *Config
    httpClient *http.Client
    mu         sync.Mutex
}

// Config holds the SDK configuration.
type Config struct {
    // ServerURL is the base URL of the Helix OTA server.
    ServerURL string

    // DeviceID is the unique identifier for this device.
    DeviceID string

    // HardwareModel identifies the device hardware (e.g.,
        "rk3588").
    HardwareModel string

    // CurrentVersion is the currently installed software version.
    CurrentVersion string

    // BuildFingerprint is the Android build fingerprint.
    BuildFingerprint string

    // CheckInterval is the duration between automatic update
        checks.
    CheckInterval time.Duration

    // ClientCert is the mTLS client certificate (PEM).
    ClientCert []byte

    // ClientKey is the mTLS client private key (PEM).
    ClientKey []byte

    // CACert is the CA certificate for server verification (PEM).
    CACert []byte

    // MaxDownloadBandwidthBps limits download bandwidth (0 =
        unlimited).
    MaxDownloadBandwidthBps int64

    // DownloadDir is the directory for storing downloaded files.
    DownloadDir string
}

// NewClient creates a new OTA SDK client with the given
        configuration.
func NewClient(config *Config) (*Client, error) {
    if err := config.validate(); err != nil {
        return nil, fmt.Errorf("invalid config: %w", err)
    }
```

```
    httpClient, err := config.buildHTTPClient()
    if err != nil {
        return nil, fmt.Errorf("failed to build HTTP client: %w",
        err)
    }

    return &Client{
        config:     config,
        httpClient: httpClient,
    }, nil
}

func (c *Config) validate() error {
    if c.ServerURL == "" {
        return fmt.Errorf("ServerURL is required")
    }
    if c.DeviceID == "" {
        return fmt.Errorf("DeviceID is required")
    }
    if c.HardwareModel == "" {
        return fmt.Errorf("HardwareModel is required")
    }
    return nil
}

func (c *Config) buildHTTPClient() (*http.Client, error) {
    tlsConfig := &tls.Config{
        MinVersion: tls.VersionTLS12,
    }

    // Configure mTLS if certificates are provided
    if len(c.ClientCert) > 0 && len(c.ClientKey) > 0 {
        cert, err := tls.X509KeyPair(c.ClientCert, c.ClientKey)
        if err != nil {
            return nil, fmt.Errorf("failed to load client
        certificate: %w", err)
        }
        tlsConfig.Certificates = []tls.Certificate{cert}
    }

    // Configure CA certificate for server verification
    if len(c.CACert) > 0 {
        caPool := x509.NewCertPool()
        if !caPool.AppendCertsFromPEM(c.CACert) {
            return nil, fmt.Errorf("failed to parse CA
        certificate")
        }
        tlsConfig.RootCAs = caPool
    }

    return &http.Client{
        Transport: &http.Transport{
            TLSClientConfig:     tlsConfig,
```

```
            MaxIdleConns:        10,
            MaxIdleConnsPerHost: 5,
            IdleConnTimeout:     90 * time.Second,
        },
        Timeout: 30 * time.Second,
    }, nil
}
```

10.3 Update Check API

```
// check.go
package helixotasdk

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

// CheckUpdateRequest is the request body for the update check
        API.
type CheckUpdateRequest struct {
    DeviceID           string `json:"device_id"`
    HardwareModel      string `json:"hardware_model"`
    CurrentBuild       string `json:"current_build"`
    CurrentVersion     string `json:"current_version"`
    BuildFingerprint   string `json:"build_fingerprint"`
    SecurityPatchLevel string `json:"security_patch_level"`
    BatteryLevel       int    `json:"battery_level"`
    AvailableStorage   int64  `json:"available_storage_bytes"`
}

// UpdateInfo describes an available update.
type UpdateInfo struct {
    UpdateID        string `json:"update_id"`
    Version         string `json:"version"`
    BuildNumber     string `json:"build_number"`
    SizeBytes       int64  `json:"size_bytes"`
    DownloadURL     string `json:"download_url"`
    SHA256Hash      string `json:"sha256_hash"`
    TargetHardware  string `json:"target_hardware"`
    MinVersion      string `json:"min_current_version"`
    Changelog       string `json:"changelog"`
    SecurityPatch   string `json:"security_patch_level"`
    ReleaseNotesURL string `json:"release_notes_url"`
    IsCritical      bool   `json:"is_critical"`
    ExpiresAt       string `json:"expires_at"`
}
```

```go
// CheckUpdateResponse is the response from the update check API.
type CheckUpdateResponse struct {
    UpdateAvailable bool       `json:"update_available"`
    UpdateInfo      *UpdateInfo `json:"update_info,omitempty"`
    PollInterval    string     `json:"poll_interval"`
    ServerTime      string     `json:"server_time"`
}

// CheckForUpdate queries the server for available updates.
func (c *Client) CheckForUpdate(ctx context.Context, req
        *CheckUpdateRequest) (*CheckUpdateResponse, error) {
    body, err := json.Marshal(req)
    if err != nil {
        return nil, fmt.Errorf("failed to marshal request: %w",
        err)
    }

    url := fmt.Sprintf("%s/v1/updates/check", c.config.ServerURL)
    httpReq, err := http.NewRequestWithContext(ctx,
        http.MethodPost, url, bytes.NewReader(body))
    if err != nil {
        return nil, fmt.Errorf("failed to create request: %w",
        err)
    }
    httpReq.Header.Set("Content-Type", "application/json")
    httpReq.Header.Set("X-Device-ID", c.config.DeviceID)
    httpReq.Header.Set("User-Agent", "HelixOTASDK/1.0.0")

    resp, err := c.httpClient.Do(httpReq)
    if err != nil {
        return nil, fmt.Errorf("request failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("server returned status %d",
        resp.StatusCode)
    }

    var result CheckUpdateResponse
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil {
        return nil, fmt.Errorf("failed to decode response: %w",
        err)
    }

    return &result, nil
}

// StartPeriodicChecks starts a background goroutine that
        periodically
// checks for updates. It calls the onUpdateAvailable callback
        when
// an update is found.
```

```
func (c *Client) StartPeriodicChecks(
    ctx context.Context,
    interval time.Duration,
    onUpdateAvailable func(*UpdateInfo),
    onError func(error),
) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            resp, err := c.CheckForUpdate(ctx,
        &CheckUpdateRequest{
                DeviceID:       c.config.DeviceID,
                HardwareModel:  c.config.HardwareModel,
                CurrentVersion: c.config.CurrentVersion,
            })
            if err != nil {
                if onError != nil {
                    onError(err)
                }
                continue
            }
            if resp.UpdateAvailable && resp.UpdateInfo != nil {
                if onUpdateAvailable != nil {
                    onUpdateAvailable(resp.UpdateInfo)
                }
            }
        }
    }
}
```

10.4 Download Manager with Resume

```
// download.go
package helixotasdk

import (
    "context"
    "crypto/sha256"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "os"
    "path/filepath"
    "strconv"
    "sync"
```

```
    "time"
)

// DownloadState tracks the state of a resumable download.
type DownloadState struct {
    URL            string    `json:"url"`
    CompletedBytes int64     `json:"completed_bytes"`
    TotalBytes     int64     `json:"total_bytes"`
    StartedAt      time.Time `json:"started_at"`
    LastActiveAt   time.Time `json:"last_active_at"`
    ChunkSize      int64     `json:"chunk_size"`
}

// DownloadProgress represents a progress update during download.
type DownloadProgress struct {
    CompletedBytes int64   `json:"completed_bytes"`
    TotalBytes     int64   `json:"total_bytes"`
    Percent        float64 `json:"percent"`
    SpeedBps       int64   `json:"speed_bps"`
    ETA            string  `json:"eta"`
}

// DownloadOption configures download behavior.
type DownloadOption func(*downloadConfig)

type downloadConfig struct {
    chunkSize       int64
    maxBandwidthBps int64
    headers         map[string]string
}

// WithChunkSize sets the download chunk size.
func WithChunkSize(size int64) DownloadOption {
    return func(c *downloadConfig) { c.chunkSize = size }
}

// WithMaxBandwidth sets the maximum download bandwidth.
func WithMaxBandwidth(bps int64) DownloadOption {
    return func(c *downloadConfig) { c.maxBandwidthBps = bps }
}

// WithHeaders adds custom HTTP headers to download requests.
func WithHeaders(headers map[string]string) DownloadOption {
    return func(c *downloadConfig) {
        for k, v := range headers {
            c.headers[k] = v
        }
    }
}

// Downloader manages resumable chunked downloads.
type Downloader struct {
    client *Client
```

```
    mu     sync.Mutex
    cancel context.CancelFunc
}

// Download downloads a file with resume support.
func (d *Downloader) Download(
    ctx context.Context,
    url string,
    targetPath string,
    expectedSHA256 string,
    onProgress func(DownloadProgress),
    opts ...DownloadOption,
) error {
    cfg := &downloadConfig{
        chunkSize:       4 * 1024 * 1024, // 4 MB
        maxBandwidthBps: d.client.config.MaxDownloadBandwidthBps,
        headers:         make(map[string]string),
    }
    for _, opt := range opts {
        opt(cfg)
    }

    statePath := targetPath + ".state"

    // Resume from previous state if available
    var state DownloadState
    if data, err := os.ReadFile(statePath); err == nil {
        json.Unmarshal(data, &state)
        if state.URL == url {
            // Valid state — resume
        } else {
            // Different URL — start fresh
            state = DownloadState{URL: url}
        }
    } else {
        state = DownloadState{
            URL:       url,
            StartedAt: time.Now(),
            ChunkSize: cfg.chunkSize,
        }
    }

    // Get total file size
    if state.TotalBytes == 0 {
        totalSize, err := d.getContentLength(ctx, url,
        cfg.headers)
        if err != nil {
            return fmt.Errorf("failed to get content length: %w",
        err)
        }
        state.TotalBytes = totalSize
    }
```

```
// Open target file for append
f, err := os.OpenFile(targetPath, os.O_CREATE|os.O_WRONLY|
    os.O_APPEND, 0644)
if err != nil {
    return fmt.Errorf("failed to open target file: %w", err)
}
defer f.Close()

// Seek to resume position
if _, err := f.Seek(state.CompletedBytes, io.SeekStart); err !
    = nil {
    return fmt.Errorf("failed to seek: %w", err)
}

ctx, d.cancel = context.WithCancel(ctx)
defer d.cancel()

buf := make([]byte, 32*1024)
startTime := time.Now()

for state.CompletedBytes < state.TotalBytes {
    select {
    case <-ctx.Done():
        d.saveState(statePath, &state)
        return ctx.Err()
    default:
    }

    chunkEnd := state.CompletedBytes + cfg.chunkSize
    if chunkEnd > state.TotalBytes {
        chunkEnd = state.TotalBytes
    }

    req, err := http.NewRequestWithContext(ctx,
    http.MethodGet, url, nil)
    if err != nil {
        return fmt.Errorf("failed to create request: %w", err)
    }
    req.Header.Set("Range",
        fmt.Sprintf("bytes=%d-%d", state.CompletedBytes,
    chunkEnd-1))
    for k, v := range cfg.headers {
        req.Header.Set(k, v)
    }

    resp, err := d.client.httpClient.Do(req)
    if err != nil {
        d.saveState(statePath, &state)
        return fmt.Errorf("download request failed: %w", err)
    }

    if resp.StatusCode != http.StatusPartialContent &&
        resp.StatusCode != http.StatusOK {
```

```
    resp.Body.Close()
    d.saveState(statePath, &state)
    return fmt.Errorf("unexpected status: %d",
resp.StatusCode)
}

chunkStart := time.Now()
written, err := io.CopyBuffer(f, resp.Body, buf)
resp.Body.Close()
if err != nil {
    d.saveState(statePath, &state)
    return fmt.Errorf("download write failed: %w", err)
}

state.CompletedBytes += written
state.LastActiveAt = time.Now()
d.saveState(statePath, &state)

// Calculate progress and speed
elapsed := time.Since(startTime).Seconds()
speedBps := int64(float64(state.CompletedBytes) / elapsed)
percent := float64(state.CompletedBytes) /
float64(state.TotalBytes) * 100

remaining := state.TotalBytes - state.CompletedBytes
var eta string
if speedBps > 0 {
    eta = (time.Duration(remaining/speedBps) *
time.Second).String()
}

if onProgress != nil {
    onProgress(DownloadProgress{
        CompletedBytes: state.CompletedBytes,
        TotalBytes:     state.TotalBytes,
        Percent:        percent,
        SpeedBps:       speedBps,
        ETA:            eta,
    })
}

// Bandwidth throttling
if cfg.maxBandwidthBps > 0 {
    expectedDuration := time.Duration(
        float64(written)/
float64(cfg.maxBandwidthBps)*float64(time.Second),
    )
    throttleWait := expectedDuration -
time.Since(chunkStart)
    if throttleWait > 0 {
        time.Sleep(throttleWait)
    }
}
```

```
    }

    // Verify SHA-256 if expected hash is provided
    if expectedSHA256 != "" {
        if err := d.verifySHA256(targetPath, expectedSHA256);
        err != nil {
            return fmt.Errorf("hash verification failed: %w", err)
        }
    }

    // Clean up state file
    os.Remove(statePath)

    return nil
}

func (d *Downloader) getContentLength(
    ctx context.Context, url string, headers map[string]string,
) (int64, error) {
    req, err := http.NewRequestWithContext(ctx, http.MethodHead,
        url, nil)
    if err != nil {
        return 0, err
    }
    for k, v := range headers {
        req.Header.Set(k, v)
    }
    resp, err := d.client.httpClient.Do(req)
    if err != nil {
        return 0, err
    }
    defer resp.Body.Close()

    contentLength := resp.ContentLength
    if contentLength <= 0 {
        // Try Content-Range header
        cr := resp.Header.Get("Content-Range")
        if cr != "" {
            // Parse "bytes 0-0/X"
            parts := strings.Split(cr, "/")
            if len(parts) == 2 {
                contentLength, _ = strconv.ParseInt(parts[1], 10,
        64)
            }
        }
    }
    if contentLength <= 0 {
        return 0, fmt.Errorf("cannot determine content length")
    }
    return contentLength, nil
}

func (d *Downloader) verifySHA256(path, expected string) error {
```

```
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()

    hash := sha256.New()
    if _, err := io.Copy(hash, f); err != nil {
        return err
    }
    computed := hex.EncodeToString(hash.Sum(nil))
    if computed != expected {
        return fmt.Errorf("SHA-256 mismatch: expected %s, got
        %s", expected, computed)
    }
    return nil
}

func (d *Downloader) saveState(path string, state *DownloadState)
        {
    data, _ := json.MarshalIndent(state, "", "  ")
    os.WriteFile(path, data, 0644)
}

// Cancel cancels an in-progress download.
func (d *Downloader) Cancel() {
    d.mu.Lock()
    defer d.mu.Unlock()
    if d.cancel != nil {
        d.cancel()
    }
}
```

10.5 Status Reporting Client

```
// report.go
package helixotasdk

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

// ReportRequest is the body of a status report to the server.
type ReportRequest struct {
    DeviceID    string `json:"device_id"`
    UpdateID    string `json:"update_id"`
    Status      string `json:"status"`
```

```
    ErrorDetail string `json:"error_detail,omitempty"`
    Timestamp   int64  `json:"timestamp"`
    CurrentSlot int    `json:"current_slot"`
    BootCount   int    `json:"boot_count_since_update"`
}

// ReportUpdateStatus sends an update status report to the server.
func (c *Client) ReportUpdateStatus(ctx context.Context, req
        *ReportRequest) error {
    body, err := json.Marshal(req)
    if err != nil {
        return fmt.Errorf("failed to marshal report: %w", err)
    }

    url := fmt.Sprintf("%s/v1/devices/%s/reports",
        c.config.ServerURL, c.config.DeviceID)
    httpReq, err := http.NewRequestWithContext(ctx,
        http.MethodPost, url, bytes.NewReader(body))
    if err != nil {
        return fmt.Errorf("failed to create request: %w", err)
    }
    httpReq.Header.Set("Content-Type", "application/json")
    httpReq.Header.Set("X-Device-ID", c.config.DeviceID)

    resp, err := c.httpClient.Do(httpReq)
    if err != nil {
        return fmt.Errorf("report request failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK && resp.StatusCode !=
        http.StatusAccepted {
        return fmt.Errorf("server returned status %d",
        resp.StatusCode)
    }

    return nil
}

// ReportCheckResult reports the outcome of an update check.
func (c *Client) ReportCheckResult(
    ctx context.Context,
    updateAvailable bool,
    serverVersion string,
) error {
    return c.ReportUpdateStatus(ctx, &ReportRequest{
        DeviceID:  c.config.DeviceID,
        Status:    "check_complete",
        Timestamp: time.Now().Unix(),
        // Include check result metadata
    })
}
```

10.6 mTLS Certificate Management

```go
// tls.go
package helixotasdk

import (
    "crypto/tls"
    "crypto/x509"
    "encoding/pem"
    "errors"
    "fmt"
    "os"
    "sync"
    "time"
)

// CertificateManager manages mTLS certificates for secure
// communication with the Helix OTA server.
type CertificateManager struct {
    mu          sync.RWMutex
    clientCert  tls.Certificate
    caCertPool  *x509.CertPool
    serverCert  *x509.Certificate
    certExpiry  time.Time
}

// NewCertificateManager creates a certificate manager from file
        paths.
func NewCertificateManager(
    clientCertPath string,
    clientKeyPath string,
    caCertPath string,
) (*CertificateManager, error) {
    cm := &CertificateManager{}

    // Load client certificate
    cert, err := tls.LoadX509KeyPair(clientCertPath,
        clientKeyPath)
    if err != nil {
        return nil,
        fmt.Errorf("failed to load client certificate: %w", err)
    }
    cm.clientCert = cert

    // Extract expiry from certificate
    if len(cert.Certificate) > 0 {
        x509Cert, err :=
        x509.ParseCertificate(cert.Certificate[0])
        if err == nil {
            cm.certExpiry = x509Cert.NotAfter
        }
    }
```

```
    // Load CA certificate
    caData, err := os.ReadFile(caCertPath)
    if err != nil {
        return nil, fmt.Errorf("failed to read CA certificate:
        %w", err)
    }
    cm.caCertPool = x509.NewCertPool()
    if !cm.caCertPool.AppendCertsFromPEM(caData) {
        return nil, errors.New("failed to parse CA certificate")
    }

    return cm, nil
}

// TLSConfig returns a *tls.Config configured for mTLS.
func (cm *CertificateManager) TLSConfig() *tls.Config {
    cm.mu.RLock()
    defer cm.mu.RUnlock()

    return &tls.Config{
        Certificates: []tls.Certificate{cm.clientCert},
        RootCAs:      cm.caCertPool,
        MinVersion:   tls.VersionTLS12,
        CipherSuites: []uint16{
            tls.TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,
            tls.TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,
            tls.TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,
            tls.TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256,
        },
    }
}

// IsCertificateExpiringSoon checks if the client certificate
// will expire within the given duration.
func (cm *CertificateManager) IsCertificateExpiringSoon(within
        time.Duration) bool {
    cm.mu.RLock()
    defer cm.mu.RUnlock()
    return time.Until(cm.certExpiry) < within
}

// RotateCertificate replaces the current client certificate with
        a new one.
// This is used when the server issues a renewed certificate.
func (cm *CertificateManager) RotateCertificate(
    clientCertPEM []byte,
    clientKeyPEM []byte,
) error {
    cm.mu.Lock()
    defer cm.mu.Unlock()

    cert, err := tls.X509KeyPair(clientCertPEM, clientKeyPEM)
    if err != nil {
```

```
        return fmt.Errorf("failed to load new certificate: %w",
        err)
    }

    if len(cert.Certificate) > 0 {
        x509Cert, err :=
        x509.ParseCertificate(cert.Certificate[0])
        if err != nil {
            return fmt.Errorf("failed to parse new certificate:
        %w", err)
        }
        cm.certExpiry = x509Cert.NotAfter
    }

    cm.clientCert = cert
    return nil
}
```

10.7 gomobile Bind Integration

The Go SDK is compiled into an Android AAR using `gomobile bind`:

```
# Build the AAR for Android
gomobile bind \
    -target=android \
    -androidapi=29 \
    -o helix-ota-sdk.aar \
    github.com/helix-ota/sdk-go

# Output: helix-ota-sdk.aar
# Contains: Java/Kotlin bindings for all exported Go types
```

Usage from Kotlin:

```
// Initialize the Go SDK from Kotlin
class GoSdkProvider @Inject constructor(context: Context) {

    private var client: Client? = null

    fun initialize(): Client {
        val config = Config().apply {
            serverURL =
        System.getProperty("ro.helix.ota.server_url")
                ?: "https://ota.helix.local/api/v1"
            deviceID = getDeviceId()
            hardwareModel = Build.HARDWARE
            currentVersion = Build.DISPLAY
            buildFingerprint = Build.FINGERPRINT
            checkInterval = 4 * 60 * 60 * 1000L // 4 hours in ms

            // Load mTLS certs from /system/etc/helix_ota/
            clientCert = loadAsset("/system/etc/helix_ota/
        client.crt")
```

```
            clientKey = loadAsset("/system/etc/helix_ota/
        client.key")
            caCert = loadAsset("/system/etc/helix_ota/ca.crt")

            downloadDir =
        context.filesDir.resolve("ota").absolutePath
            maxDownloadBandwidthBps = 10 * 1024 * 1024 // 10 MB/s
        }

        client = Client(config)
        return client!!
    }

    private fun loadAsset(path: String): ByteArray {
        return File(path).readBytes()
    }

    private fun getDeviceId(): String {
        // Use stable device identifier
        return Settings.Secure.getString(
            context.contentResolver,
            Settings.Secure.ANDROID_ID
        )
    }
}
```

Appendix A: Error Code Reference

Code	Category	Description
1001	Verify	Missing update info
1002	Verify	ZIP file corrupt
1003	Verify	SHA-256 hash mismatch
1004	Verify	RSA signature invalid
1005	Verify	payload_properties.txt invalid
1006	Verify	Device incompatible
1007	Verify	Unexpected verification error
2001	Download	Network error
2002	Download	HTTP error (status code)
2003	Download	Insufficient disk space
2004	Download	Write error
2005	Download	Cancelled by user
3001	Install	update_engine bind failed
3002	Install	update_engine applyPayload failed
3003	Install	update_engine reported error (sub-code from UpdateEngineError)

Code	Category	Description
3004	Install	Boot slot switch failed
3005	Install	Post-install verification failed
3006	Install	Merge phase failed
4001	Report	Server unreachable
4002	Report	Authentication failed

Appendix B: update_engine Status Codes

Constant	Value	Description
`UPDATE_STATUS_IDLE`	0	No update in progress
`UPDATE_STATUS_CHECKING_FOR_UPDATE`	1	Checking for update
`UPDATE_STATUS_UPDATE_AVAILABLE`	2	Update available
`UPDATE_STATUS_DOWNLOADING`	3	Downloading update
`UPDATE_STATUS_VERIFYING`	4	Verifying update
`UPDATE_STATUS_FINALIZING`	5	Finalizing update
`UPDATE_STATUS_UPDATED_NEED_REBOOT`	6	Update applied, needs reboot
`UPDATE_STATUS_REPORTING_ERROR_EVENT`	7	Reporting error
`UPDATE_STATUS_ATTEMPTING_ROLLBACK`	8	Attempting rollback
`UPDATE_STATUS_DISABLED`	9	Update engine disabled
`UPDATE_STATUS_CLEANUP_PREVIOUS_UPDATE`	10	Cleaning up previous update

Appendix C: update_engine Error Codes

Constant	Value	Description
`SUCCESS`	0	Operation succeeded
`ERROR`	1	General error
`FILESYSTEM_COPIER_ERROR`	4	Filesystem copy failed
`POST_INSTALL_RUNNER_ERROR`	5	Post-install script failed
`PAYLOAD_MISMATCHED_TYPE_ERROR`	6	Payload type doesn’t match
`INSTALL_DEVICE_OPEN_ERROR`	7	Cannot open install device
`KERNEL_DEVICE_OPEN_ERROR`	8	

Constant	Value	Description
		Cannot open kernel device
DOWNLOAD_TRANSFER_ERROR	9	Download transfer failed
PAYLOAD_HASH_MISMATCH_ERROR	10	Payload hash doesn’t match
PAYLOAD_SIZE_MISMATCH_ERROR	11	Payload size doesn’t match
DOWNLOAD_PAYLOAD_VERIFICATION_ERROR	12	Payload verification failed
SIGNED_DELTA_PAYLOAD_EXPECTED_ERROR	13	Expected signed delta payload
DOWNLOAD_PAYLOAD_PUBKEY_VERIFICATION_ERROR	14	Public key verification failed
NOT_ENOUGH_SPACE	28	Insufficient space for update
DEVICE_CORRUPTED	61	Device is corrupted

Document generated for Helix OTA 1.0.0-MVP. For questions, contact the Helix OTA team.